P2P File Share

The complete guide: how it works, why it is built this way, what changed, and where it can go next

WebRTC DataChannel · FastAPI signaling · coturn STUN/TURN · Docker

Who this is for: anyone who wants to understand this project end to end - to maintain it, extend it, present it, or learn WebRTC from a small real codebase. No prior WebRTC knowledge is assumed; each idea is introduced before the code that uses it.

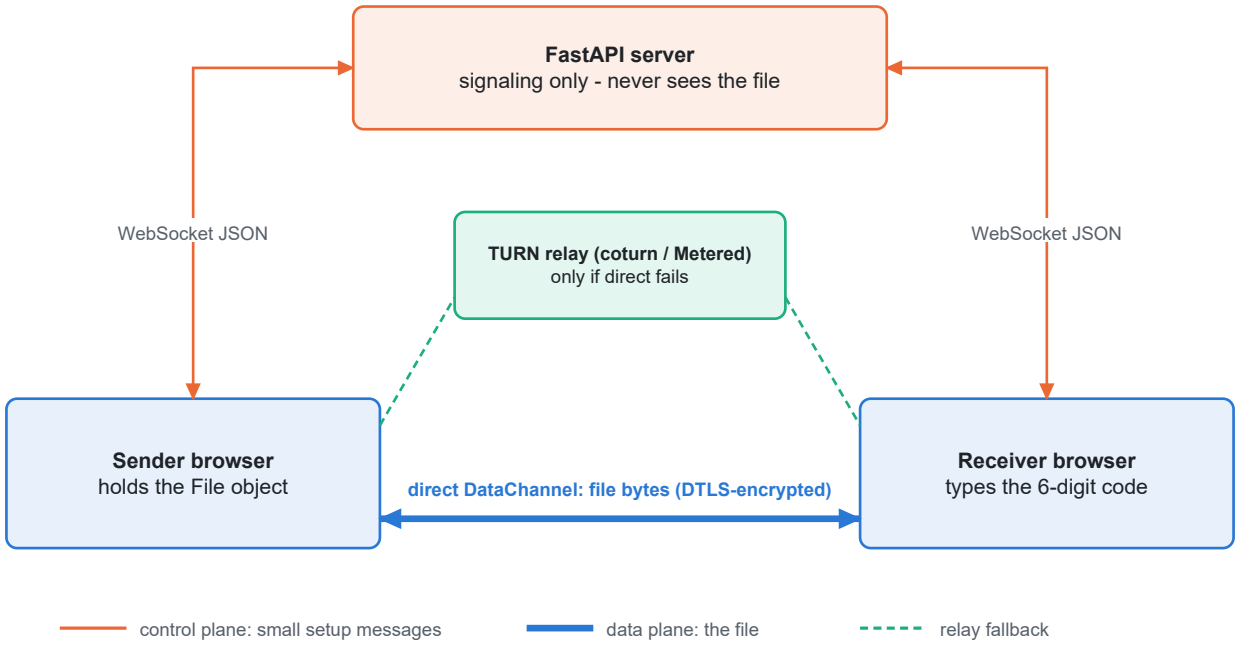


Figure 1. The whole system on one page. The server only introduces the two browsers; the file travels directly between them (or through a TURN relay when a direct path is impossible).

What you will learn	Where
Why two browsers need a server to meet, but not to talk	Chapters 2-4
Every function in `main.py` and `app.js`, line by line, with flowcharts	Chapters 5-6
The exact messages on the wire	Chapter 7
Docker, coturn, HTTPS and running it over the internet	Chapters 8-9
Security model, changes made so far, testing, troubleshooting	Chapters 10-13
Future roadmap, exercises, quiz and glossary	Chapters 14-15

Contents

1. Introduction . . . 5

1.1 The project at a glance . . . 5

1.2 How to read this guide . . . 5

2. Core concepts: a WebRTC primer . . . 6

2.1 The problem: browsers cannot simply dial each other . . . 6

2.2 Signaling . . . 6

2.3 SDP: the offer and the answer . . . 6

2.4 ICE, STUN and TURN . . . 6

2.5 DataChannel, SCTP and DTLS . . . 7

3. Architecture . . . 8

3.1 Components . . . 8

3.2 Design principles visible in the code . . . 8

4. End-to-end walkthrough . . . 10

5. The server: main.py . . . 12

5.1 Configuration . . . 12

5.2 The app, static files and the rooms table . . . 12

5.3 send_safe() . . . 12

5.4 GET / and GET /config . . . 13

5.5 WS /ws/{room_id}: the signaling relay . . . 14

5.6 Room lifecycle . . . 16

5.7 The subtle details . . . 16

5.8 Close codes . . . 16

6. The client: static/app.js . . . 18

6.1 State and constants . . . 18

6.2 Page-load warm-ups . . . 18

6.3 Starting a transfer . . . 18

6.4 join(): opening the signaling socket . . . 20

6.5 The message chain: why order matters . . . 20

6.6 handle(): the signaling state machine . . . 21

6.7 createPeer() and forcing the relay . . . 22

6.8 findRoute(): direct or relay? . . . 22

6.9 Sending: setupChannel() and sendFile() . . . 22

6.10 The DataChannel mini-protocol . . . 24

6.11 Receiving: receiveInto() and save() . . . 25

6.12 Cleanup and the "stale callback" pattern . . . 25
6.13 The page: index.html . . . 25

7. Wire protocol reference . . . 27

7.1 Signaling messages (WebSocket, JSON text) . . . 27
7.2 DataChannel messages . . . 27
7.3 HTTP endpoints . . . 27

8. Deployment: Docker, coturn and HTTPS . . . 28

8.1 Two containers . . . 28
8.2 The web image (Dockerfile) . . . 28
8.3 docker-compose.yml . . . 29
8.4 coturn flags . . . 29
8.5 HTTPS and the certificate . . . 29
8.6 Running it . . . 30
8.7 Over the internet . . . 30

9. Networking in practice: LAN vs internet . . . 31

9.1 What speed to expect . . . 32

10. Security model . . . 33

10.1 Threats and current mitigations . . . 33
10.2 How likely are code collisions? . . . 33

11. Changes made: how the code evolved . . . 35

11.1 Change log . . . 35
11.2 Where EXPLANATION.md is out of date . . . 36

12. Testing . . . 37

12.1 What has been tested . . . 37
12.2 Manual test checklist . . . 37
12.3 A server test you could add . . . 37

13. Troubleshooting . . . 38

14. Future aspects: where this can go . . . 40

14.1 Stream large files to disk . . . 40
14.2 Integrity check . . . 40
14.3 Multiple files and folders . . . 40
14.4 Several receivers at once . . . 41
14.5 Resume after a disconnect . . . 41
14.6 Security hardening . . . 41

14.7 Scale and operations . . . 41
14.8 User experience . . . 41
14.9 Housekeeping . . . 41

15. Learning aids . . . 42

15.1 Hands-on exercises . . . 42
15.2 Quiz . . . 42
15.3 Glossary . . . 43

Appendix A. Full source code . . . 44

A.1 main.py . . . 44
A.2 static/app.js . . . 46
A.3 static/index.html . . . 50
A.4 docker-compose.yml . . . 51
A.5 Dockerfile . . . 51
A.6 .env.example . . . 51
A.7 requirements.txt and .dockerignore . . . 51

1. Introduction

P2P File Share sends a file from one browser to another using a 6-digit code. The sender picks a file and gets a code; the receiver types the code and the download starts. The file travels **browser to browser** over a WebRTC DataChannel. The server never receives a single byte of it - it only helps the two browsers find each other.

1.1 The project at a glance

Item	Value
Source files	`main.py` (125 lines), `static/app.js` (268 lines), `static/index.html` (42 lines), `Dockerfile`, `docker-compose.yml`, `.env`
Server	Python 3.12, FastAPI, uvicorn (HTTPS + WSS on port 8000), python-dotenv
Client	Plain JavaScript, no framework, no build step, no npm packages
NAT traversal	coturn 4.18.0 (STUN + TURN, port 3478, relay ports 49160-49200), Google public STUN, optional Metered hosted TURN
Transfer unit	64 KB chunks (or smaller if the peer advertises a lower `maxMessageSize`)
Flow control	pause reading above 1 MB queued, resume at 512 KB
Code	6 random digits from `crypto.getRandomValues` - 1,000,000 possible codes
Encryption	Always on: DTLS for the file, TLS (self-signed) for the page and signaling

1.2 How to read this guide

- **New to WebRTC?** Read chapters 2 to 4 in order. They build the mental model the rest depends on.
- **Want to change the code?** Chapters 5 and 6 explain every function, with the reason behind each guard.
- **Deploying or debugging?** Chapters 8, 9 and 13.
- **Teaching it?** Each chapter ends in a short "key idea" box; chapter 15 has exercises, a quiz with answers and a glossary. Appendix A contains the full source code with line numbers.

> **A note on sources**
> This guide is written from the current source code. EXPLANATION.md in the repository was the starting point, but in a few places the code has moved on since it was written (for example the back-pressure limits). Where they differ, this guide follows the code, and chapter 11 lists every difference.

2. Core concepts: a WebRTC primer

Before reading any code you need six ideas. Each solves a specific problem that appears when two browsers try to talk directly.

2.1 The problem: browsers cannot simply dial each other

A web page normally talks to a server, which has a public address. Two **browsers** have no such address. Each sits behind a router doing **NAT** (Network Address Translation): the laptop thinks it is `192.168.1.20`, but the outside world sees the router's public IP and some port the router picked. Nobody outside can start a connection to it, and neither browser even knows its own public address.

WebRTC solves this in three steps: **exchange setup information** through some server (signaling), **discover candidate addresses** (ICE, with help from STUN), and **fall back to a relay** when nothing direct works (TURN).

2.2 Signaling

Signaling is passing setup messages between the two browsers **before** they are connected. WebRTC deliberately does not specify how - any channel works (a WebSocket, HTTP polling, even copy-paste). This project uses a WebSocket at `/ws/{room}` on the FastAPI server. The server is a dumb mailbox: it forwards each message to the other browser in the same room without looking inside.

> **Analogy**
> Signaling is a receptionist who passes two strangers each other's phone numbers. After that, they call each other directly; the receptionist does not hear the call.

2.3 SDP: the offer and the answer

SDP (Session Description Protocol) is a text blob that says "here is what I want to open (a data channel), and here is my encryption fingerprint". One side creates an **offer**, the other replies with an **answer**. Each side stores its own description with `setLocalDescription` and the other side's with `setRemoteDescription`. Exactly one side must offer; if both offer at once ("glare") the negotiation breaks. Section 5.7 shows how the server prevents that.

2.4 ICE, STUN and TURN

An **ICE candidate** is one possible address where a browser might be reachable: IP + port + type. There are three types you will meet:

Candidate type	Where it comes from	Works when
`host`	The device's own network interface (e.g. 192.168.1.20)	Both devices are on the same network
`srflx` (server reflexive)	A **STUN** server replies "from here you look like 49.36.x.x:51234"	Both NATs allow UDP hole punching (most home routers)
`relay`	A **TURN** server allocates a port and relays packets for you	Always, if the TURN server is reachable - at the cost of its bandwidth

Browsers gather candidates and send each one to the peer as soon as it is found ("trickle ICE"). ICE then tests candidate pairs and selects the best one that works. STUN is only used during setup; TURN, if chosen, carries all the (still encrypted) data.

2.5 DataChannel, SCTP and DTLS

An **RTCDataChannel** is a peer-to-peer pipe for arbitrary messages - strings or binary. By default it is **reliable and ordered**, like TCP, so every chunk arrives exactly once and in order. Underneath, messages are framed by **SCTP** and encrypted by **DTLS** (TLS for UDP). The DTLS keys are checked against the fingerprint in the SDP, which is why the data is encrypted end to end even when it passes through TURN.

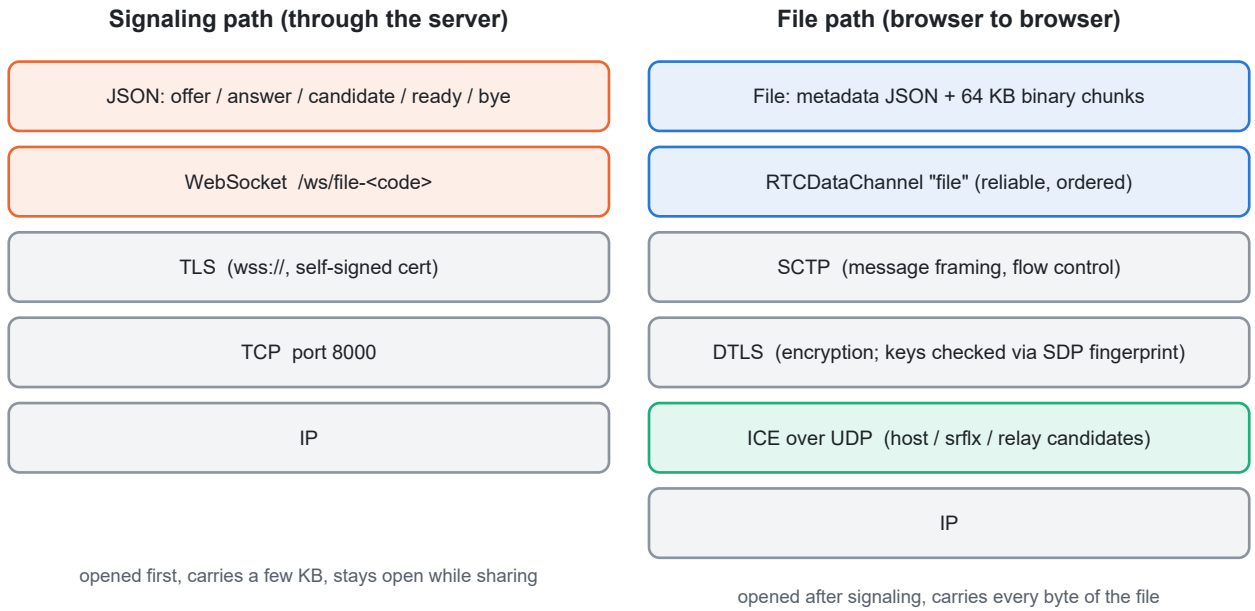


Figure 2. The two protocol stacks. The left one is used for a few kilobytes of setup; the right one carries the file.

> **Key idea**
> There are two separate channels: a **control plane** (WebSocket through the server, tiny JSON messages) and a **data plane** (DataChannel, peer to peer, the actual file). Keep them apart in your head and the rest of the code becomes easy to follow.

3. Architecture

3.1 Components

Component	Runs where	Responsibility
`static/index.html`	Browser	The page: a Send box (file input, big code, status), a Receive box (code input, button, status) and a progress bar
`static/app.js`	Browser	All WebRTC and transfer logic: signaling client, peer connection, chunked sending with back-pressure, receiving and saving
`main.py`	`web` container	Serves the page, hands out the ICE server list at `/config`, relays signaling messages at `/ws/{room}`
coturn	`coturn` container	Our own STUN + TURN server on port 3478
Metered (optional)	Cloud	Hosted TURN reachable from anywhere; used instead of coturn when `METERED_TURN_URL` is set
Google STUN	Cloud	`stun.l.google.com:19302`, a free backup STUN server

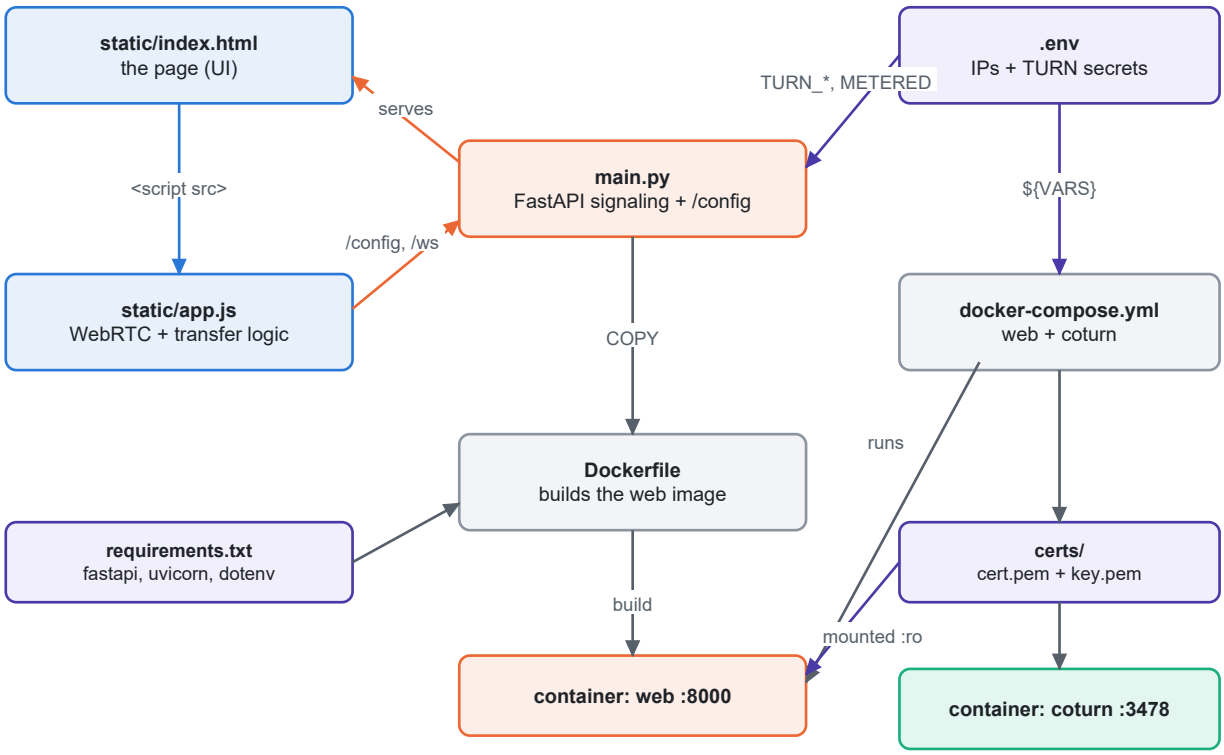


Figure 3. How the files relate. Blue = browser code, orange = server, violet = configuration and secrets.

3.2 Design principles visible in the code

- **The server knows as little as possible.** It never parses signaling payloads and never sees file data, which keeps it at ~125 lines and makes it trustworthy by construction.
- **Configuration lives in one place.** IPs and TURN passwords come from `.env`; the browser fetches them from `/config` instead of hardcoding them.
- **No dependencies where the platform suffices.** No frontend framework, no WebRTC wrapper library; the browser APIs are used directly.
- **Every async callback checks it is still current** (`conn !== pc`, `socket !== ws`, `id !== joinId`). This is the main defence against race conditions and is explained in section 6.12.

- **Known limits are marked, not hidden.** The in-memory receive buffer carries a `ponytail:` comment naming its ceiling and the upgrade path.

4. End-to-end walkthrough

This chapter follows one transfer from page load to download. Read the diagram top to bottom; the numbered list below explains each phase.

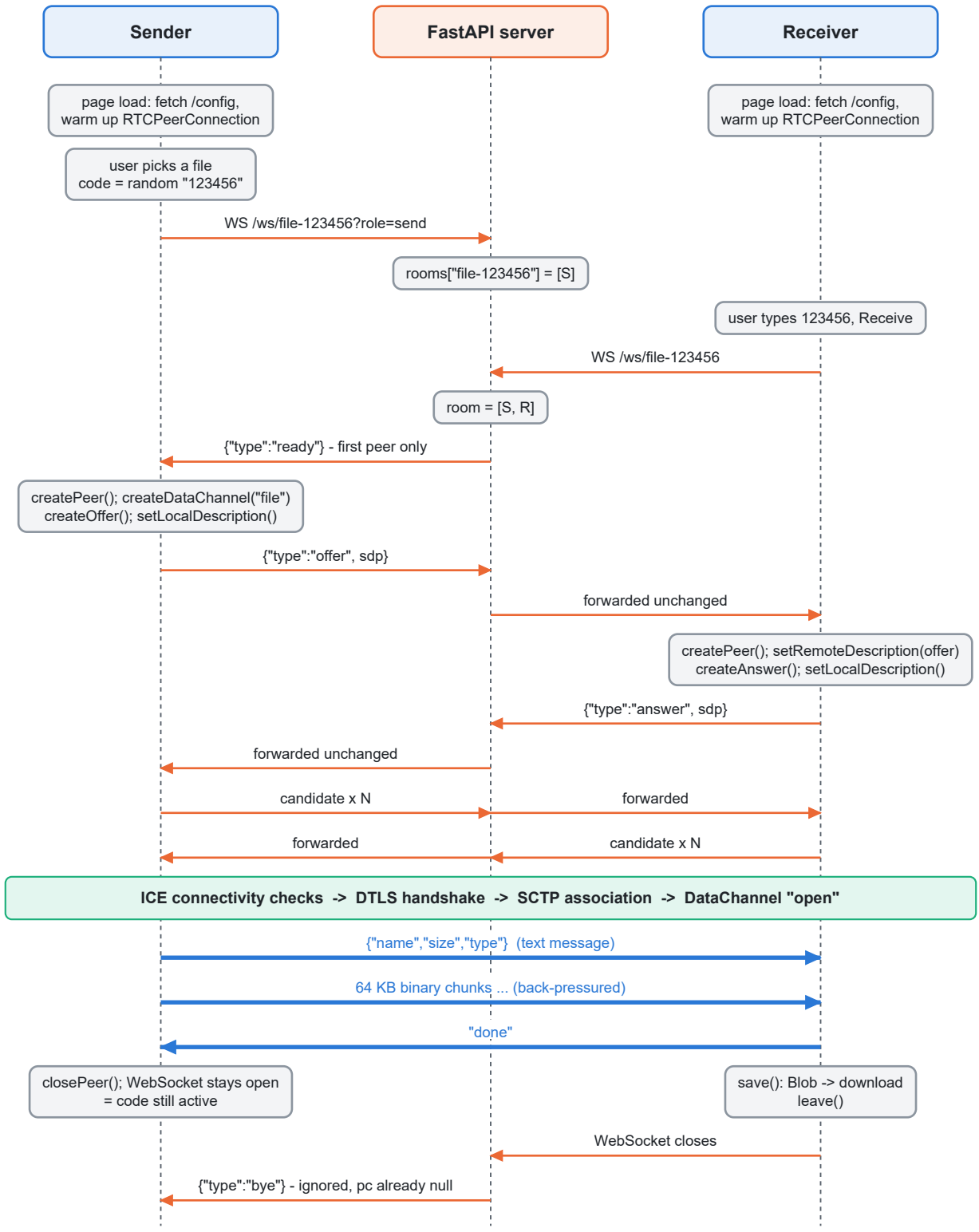


Figure 4. One complete transfer. Orange arrows go through the signaling server; blue arrows travel directly over the DataChannel.

1. **Page load.** Both tabs start fetching `/config` immediately (`configReady`) and create and close a throwaway `RTCPeerConnection`, so the browser's one-time WebRTC start-up cost is paid while the user is still choosing a file or typing.

2. **Sender picks a file.** `share()` draws a random 6-digit code, shows it, and opens `/ws/file-<code>?role=send`. The server creates the room. If the code happens to be taken, the server closes with 4409 and the page silently draws another.
3. **Receiver enters the code.** `join()` opens `/ws/file-<code>` without a role. The server adds it to the room (or closes with 4404 if no such room exists).
4. **Server says "ready".** With two sockets in the room, the server sends `{"type":"ready"}` to the first one only - always the sender. That makes the sender the **offerer**.
5. **Offer.** The sender creates the peer connection, creates the DataChannel `"file"` (before the offer, so the SDP includes a data section), creates the offer and sends it.
6. **Answer.** The receiver applies the offer, creates the answer and sends it back.
7. **Trickle ICE.** Each side sends every candidate it discovers, as it discovers it.
8. **Connection.** ICE picks a working pair, DTLS completes, the DataChannel opens. The receiver gets it via `ondatachannel`. `findRoute()` labels the path as direct or relay.
9. **Transfer.** The sender sends one JSON metadata string, then 64 KB binary chunks, pausing whenever more than 1 MB is queued. The receiver collects the chunks and updates the progress bar and speed.
10. **Finish.** When the receiver has `meta.size` bytes it saves the file and replies `"done"`. The sender closes the peer connection but **keeps its WebSocket**, so the code stays valid for the next receiver. The receiver's channel closes, it calls `leave()`, and the server tells the sender `bye` (ignored, because the sender has already closed its peer connection).

5. The server: main.py

The server has three jobs: serve the page, hand out ICE servers, and relay signaling messages. All three fit in one file.

5.1 Configuration

```
BASE = Path(__file__).parent
load_dotenv(BASE / ".env")

TURN_HOST = os.getenv("TURN_HOST", "127.0.0.1")
TURN_USER = os.getenv("TURN_USER", "webrtc")
TURN_PASS = os.getenv("TURN_PASS", "webrtc")
METERED_TURN_URL = os.getenv("METERED_TURN_URL")
metered_cache: tuple[float, list] = (0.0, [])  # (fetched at, iceServers)
```

- `BASE` is the folder containing `main.py`; every path is built from it, so the server works from any working directory.
- `load_dotenv` reads `.env` into environment variables. Docker Compose reads the same file, so the server and coturn always agree on the TURN user and password.
- `metered_cache` remembers the last Metered answer and when it was fetched (section 5.4).

Variable	Used by	Meaning
`TURN_HOST`	main.py	Address browsers use to reach coturn (LAN IP, or public IP over the internet)
`EXTERNAL_IP`	coturn	Address coturn writes into its relay candidates (normally the same as TURN_HOST)
`TURN_USER` / `TURN_PASS`	both	TURN long-term credentials
`METERED_TURN_URL`	main.py	Optional. Metered credentials URL including `?apiKey=`; when set, `/config` hands out Metered TURN instead of coturn

5.2 The app, static files and the rooms table

```
app = FastAPI()
app.mount("/static", StaticFiles(directory=BASE / "static"), name="static")

rooms: dict[str, list[WebSocket]] = {}  # room id -> the (max 2) sockets in it
```

Everything in `static/` is served under `/static/...`. `rooms` is plain process memory: restarting the server forgets every code (senders see "disconnected from server" and pick the file again). That is fine for a single uvicorn process, and it is why the server must not be run with several workers (each would have its own `rooms`) - see section 14.

5.3 send_safe()

```
async def send_safe(ws: WebSocket, text: str):
    # the other peer may be disconnecting right now; that must not kill this connection
    try:
        await ws.send_text(text)
    except Exception:
        pass
```

Every message the server sends to a *peer* goes through this. If the other browser disconnects at the same moment, `send_text` raises. Without the `try`, that exception would end **your** handler and disconnect **you**. With it, the failed send is ignored, and the dead peer's own handler cleans up after itself.

5.4 GET / and GET /config

`/` returns `static/index.html`. `/config` returns exactly the object the browser passes to `new RTCPeerConnection(...)`:

```
{"iceServers": [
  {"urls": "stun:stun.l.google.com:19302"},          # free backup STUN
  {"urls": "stun:TURN_HOST:3478"},                    # coturn as STUN
  {"urls": ["turn:TURN_HOST:3478?transport=udp",      # coturn as TURN (UDP preferred)
            "turn:TURN_HOST:3478?transport=tcp"],     # TCP for networks that block UDP
   "username": TURN_USER, "credential": TURN_PASS}
]}
```

When `METERED_TURN_URL` is set, `/config` first tries Metered's API, which returns a ready-made list with fresh credentials. The API key stays on the server. The logic is in `metered_ice_servers()`:

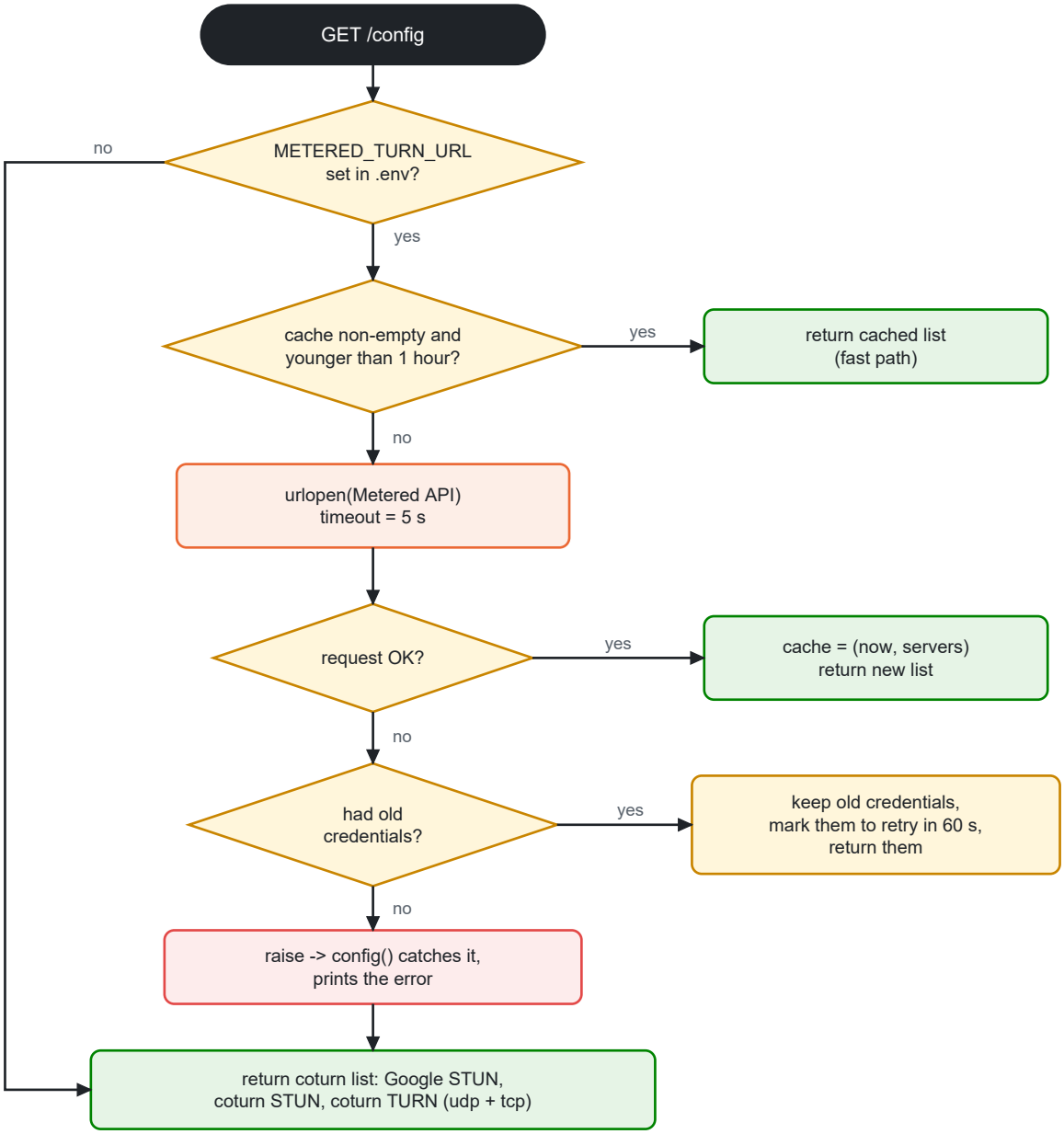


Figure 5. Decision flow of GET /config, including the one-hour Metered cache and the fallback to coturn.

- **Why cache?** The Metered API takes seconds to answer, and its credentials do not change per request. Reusing them for an hour makes `/config` instant.
- **Why "retry in 60 s"?** If a refresh fails but old credentials exist, they are kept and the timestamp is set to `time.time() - 3600 + 60`, which makes the cache look stale again in one minute. One outage does not cause a

request storm, and old credentials are better than none.

- **Why is it safe to block?** `config()` is a plain `def`, not `async def`, so FastAPI runs it in a worker thread. The blocking `urllib` call does not freeze the WebSocket relay.

5.5 WS /ws/{room_id}: the signaling relay

```
@app.websocket("/ws/{room_id}")
async def signaling(websocket: WebSocket, room_id: str, role: str = ""):
    await websocket.accept()
    if role == "send":
        if room_id in rooms:
            await websocket.close(code=4409)  # code already taken
            return
        peers = rooms[room_id] = []
    elif room_id in rooms:
        peers = rooms[room_id]
    else:
        await websocket.close(code=4404)      # expired or wrong code
        return
    if len(peers) >= 2:
        await websocket.send_text(json.dumps({"type": "full"}))
        await websocket.close(code=1008)
        return
    peers.append(websocket)
    if len(peers) == 2:
        await send_safe(peers[0], json.dumps({"type": "ready"}))
    try:
        while True:
            text = await websocket.receive_text()
            for peer in peers:
                if peer is not websocket:
                    await send_safe(peer, text)   # forward as-is
    except WebSocketDisconnect:
        pass
    finally:
        peers.remove(websocket)
        if rooms.get(room_id) is peers and (role == "send" or not peers):
            del rooms[room_id]
        for peer in peers:
            await send_safe(peer, json.dumps({"type": "bye"}))
```

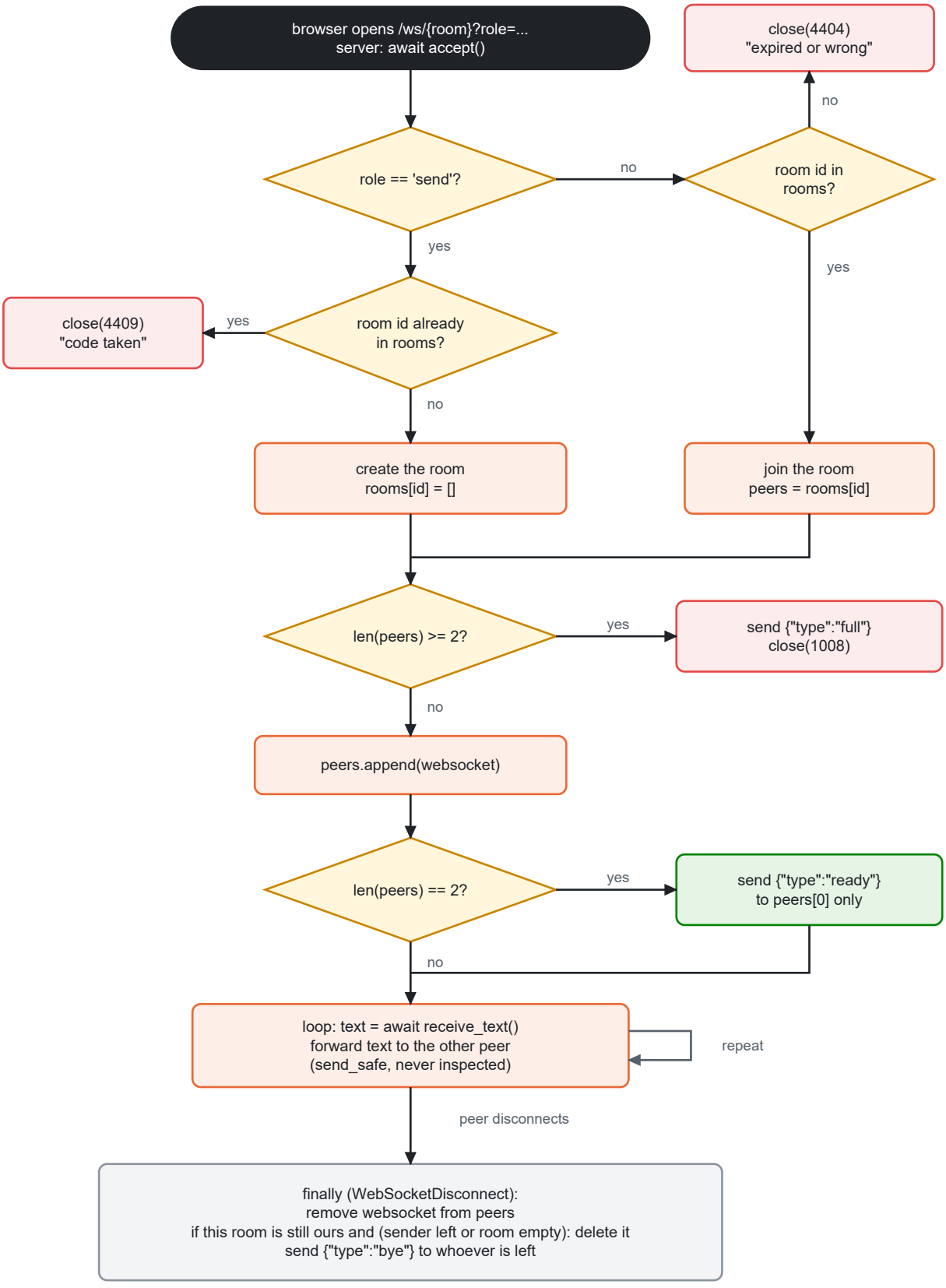
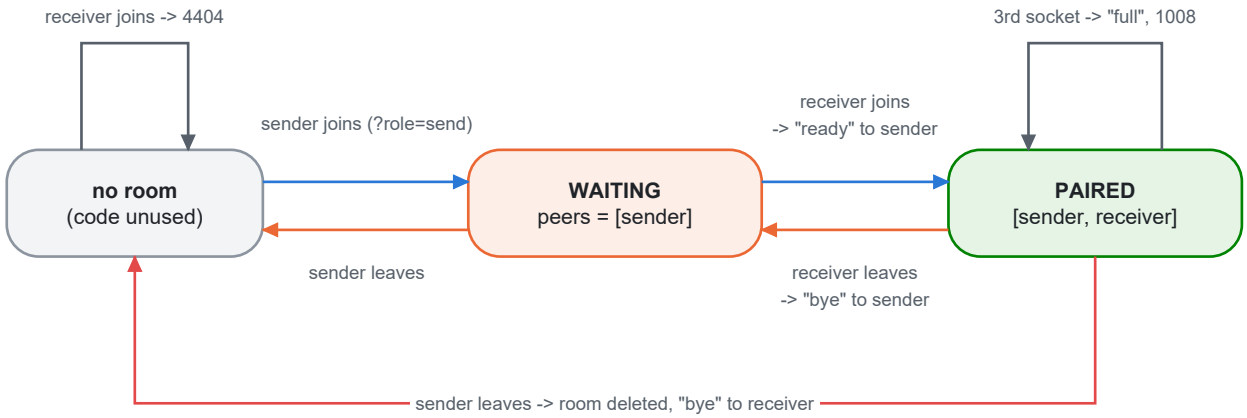


Figure 6. The signaling handler as a flowchart. Red boxes reject the connection; the loop forwards every message unchanged until the socket disconnects.

5.6 Room lifecycle



The sender owns the code: the room lives exactly as long as the sender's WebSocket.

Figure 7. States of one room (one code). The sender creates it and its departure destroys it.

5.7 The subtle details

"ready" goes to the first socket only

Exactly one side must create the offer. If both did, two offers would cross ("glare") and negotiation would fail. The server breaks the symmetry by telling only `peers[0]` - which, because only senders can create rooms, is always the sender.

Why the room check needs no lock

Between `if room_id in rooms` and `rooms[room_id] = []` there is no `await`. In asyncio, code only switches tasks at an `await`, so these lines run atomically: two senders can never both create the same room.

`rooms.get(room_id) is peers` - the identity check

Imagine sender A leaves code 123456 (room deleted), and sender B immediately draws 123456 again (new room, new list). If A's old receiver disconnects afterwards, its cleanup must not delete B's room. Comparing the list object with `is` (not the name) guarantees each handler only deletes the room it actually joined.

"bye" on every departure

Whoever is left in the room gets `{"type":"bye"}`. The client decides what it means: a sender keeps waiting on the code; a receiver knows the code is dead.

5.8 Close codes

Code	Sent when	What the page does
4409	A sender asks for a code that already has a room	Silently draws a new code (`share(file)`)
4404	A receiver asks for a code with no room	Shows "code expired ... or wrong"
1008	A third socket joins a full room (after a `full` message)	Shows "code busy, someone is receiving it"
other	Server restarted, network lost	Shows "disconnected from server"; a sender's code is cleared

> **Key idea**
> The server is a **room-scoped message forwarder** with three rules: senders own codes, at most two sockets per room, and the first socket offers. Everything else is the browsers' business.

6. The client: static/app.js

6.1 State and constants

Name	Holds	Notes
`ws`	WebSocket to the signaling server	Set to `null` before closing it ourselves
`pc`	RTCPeerConnection to the other browser	`null` when no transfer is active
`iceServers`	STUN/TURN list from `/config`	Copied in on each `join()`
`route`	"direct, same network", "direct" or "relay"	Filled by `findRoute()`, shown in status
`chain`	A Promise queue	Runs signaling messages one at a time, in order
`file`	The File being shared, or `null`	**Decides the role:** set = sender, null = receiver
`statusEl`	The active status line	Send box or Receive box
`joinId`	A generation counter	Bumped by `leave()` so a slow `join()` cannot revive an old code
`CHUNK`	64 * 1024	Bytes per DataChannel message
`HIGH_WATER`	1024 * 1024	Pause reading when this much is queued; resume at half

Small helpers: `$(id)` is `getElementById`; `status(t)` writes the active status line; `mb(n)` formats megabytes; `progress(done, total)` shows and fills the bar; `send(obj)` JSON-encodes a message and sends it only if the socket is open.

6.2 Page-load warm-ups

```
const loadConfig = () => fetch("/config").then((r) => r.json()).then((c) => c.iceServers);
let configReady = loadConfig();
setTimeout(() => new RTCPeerConnection().close(), 0);
```

Both lines exist purely for speed. The ICE list starts downloading the moment the page opens, so pressing Send or Receive does not wait for it. The first `RTCPeerConnection` in a browser can take seconds to create (WebRTC sets up its networking stack); creating and immediately closing one right after render moves that delay to a moment when the user is busy anyway.

6.3 Starting a transfer

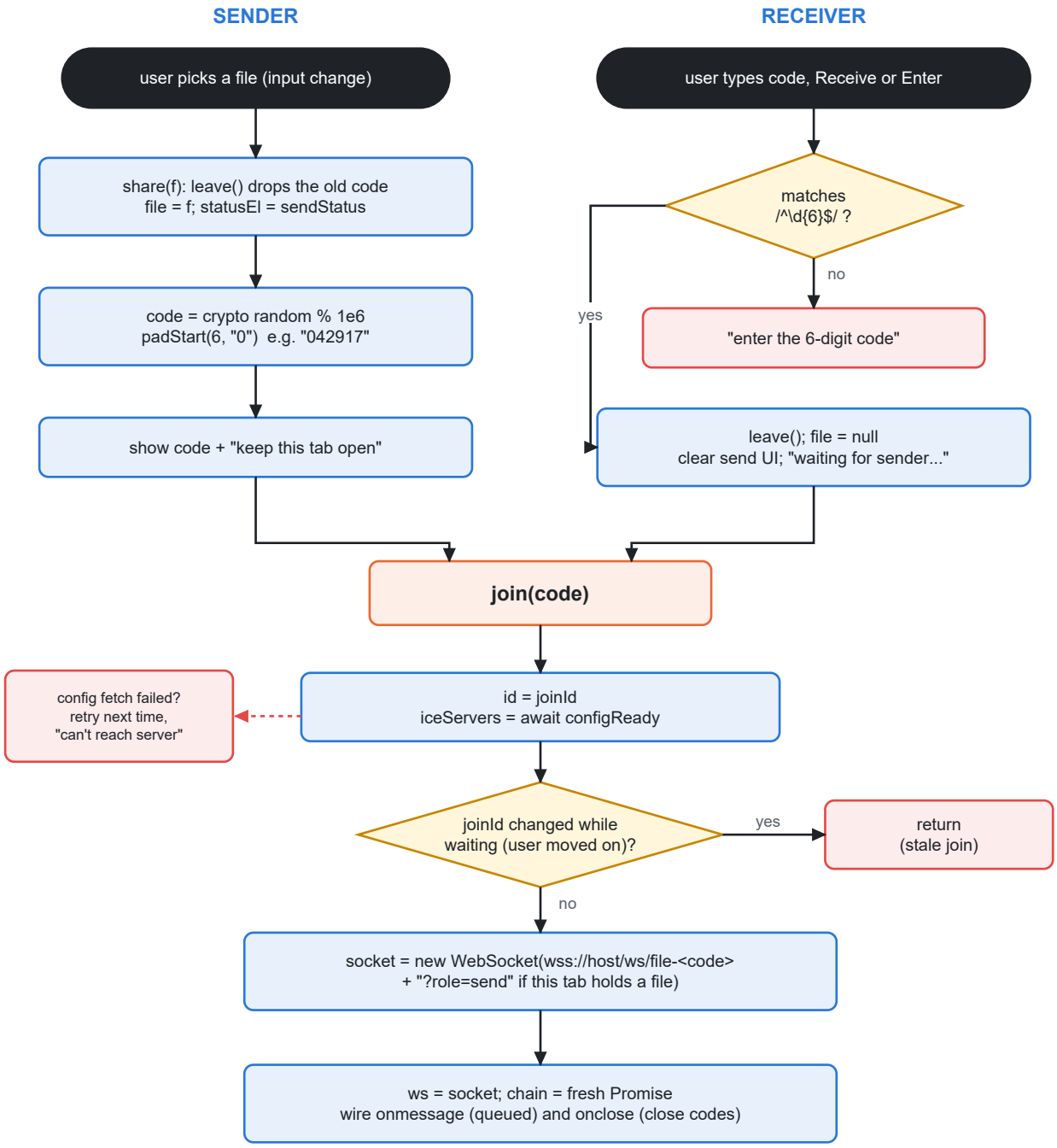


Figure 8. What happens when the sender picks a file and when the receiver presses Receive. Both paths end in join(code).

- **Sender:** `share(f)` first calls `leave()`, which drops any previous code on the server immediately. The code is `crypto.getRandomValues(...) % 1e6`, zero-padded to six digits - easy to type on a phone.
- **Receiver:** the input must match `/^\d{6}$/`. `inputmode="numeric"` in the HTML brings up the number pad on phones, and Enter in the box clicks Receive.
- A tab is either a sender or a receiver at a time; starting one role resets the other's UI.

6.4 join(): opening the signaling socket

```
async function join(code) {
    const id = joinId;
    try { iceServers = await configReady; }
    catch (err) { configReady = loadConfig(); status("can't reach server"); return; }
    if (id !== joinId) return;            // user picked another file / code meanwhile
    route = "";
    const proto = location.protocol === "https:" ? "wss" : "ws";
    const socket = new WebSocket(`${proto}://${location.host}/ws/file-${code}${file ? "?role=send" : ""}`);
    ws = socket;
    chain = Promise.resolve();
    socket.onmessage = (e) => { const msg = JSON.parse(e.data);
        chain = chain.then(() => socket === ws && handle(msg)).catch(console.error); };
    socket.onclose = (e) => {
        if (socket !== ws) return;        // we closed it ourselves
        ws = null; closePeer();
        if (e.code === 4409) share(file); // random code taken: pick another
        else if (e.code === 4404) status(EXPIRED);
        else if (e.code === 1008) status("code busy, someone is receiving it, try again in a moment");
        else status("disconnected from server");
        if (file && e.code !== 4409) $("code").textContent = "";
    };
}
```

- An HTTPS page must use `wss://`. `location.host` connects back to whatever address the page was opened with, so no IP is hardcoded in JavaScript.
- If `/config` failed, `configReady` is replaced with a fresh attempt, so the next click retries.
- A **fresh chain** on every join means a queue stuck on an old, closed connection (WebRTC promises on a closed `pc` may never settle) cannot block the new one.

6.5 The message chain: why order matters

`handle()` is `async`. Messages can arrive faster than they are processed: an `offer` and then a `candidate` right behind it. Without a queue, the candidate handler would run while the offer handler is still inside `await setRemoteDescription(...)`, and `addIceCandidate` would fail because there is no remote description yet.



Figure 9. The same two messages without and with the promise chain.

`chain = chain.then(...)` appends each message to the end of a promise queue, so handlers run strictly one after another. `socket === ws` drops leftovers from an old socket, and `.catch(console.error)` keeps one failing message from breaking the queue for all later ones.

6.6 handle(): the signaling state machine

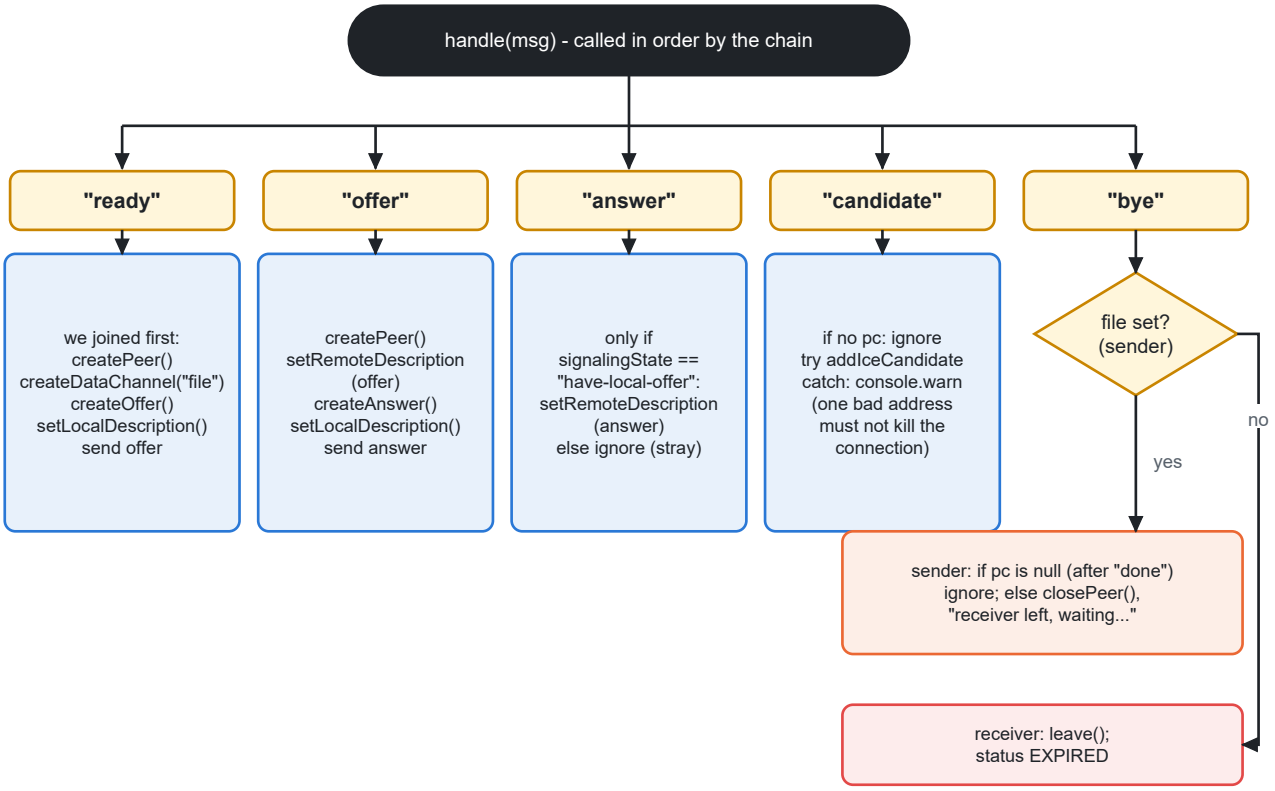
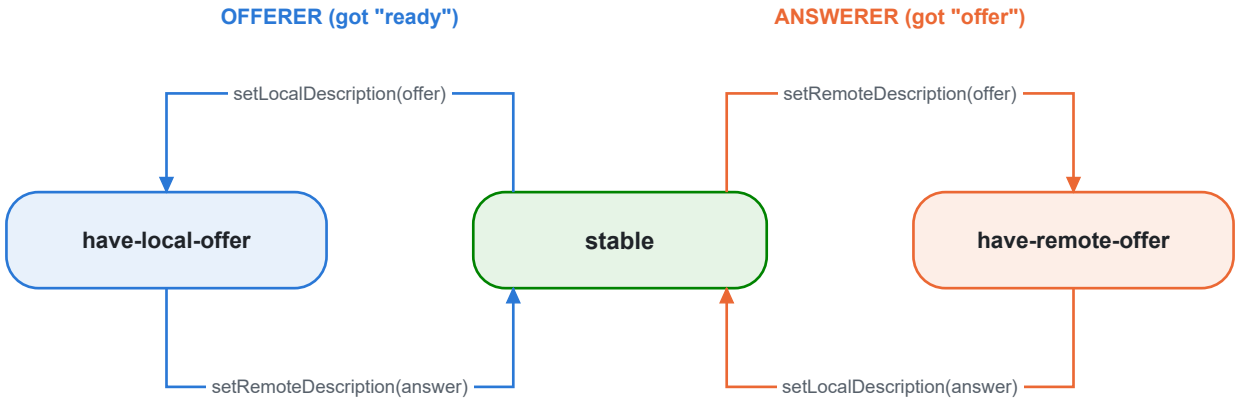


Figure 10. What each signaling message does.

The guards in each branch map to the RTCPeerConnection **signaling state**. An offerer goes `stable -> have-local-offer -> stable`; an answerer goes `stable -> have-remote-offer -> stable`:



An "answer" is applied only in have-local-offer, so a duplicate answer is ignored.

Figure 11. RTCPeerConnection.signalingState for the two roles.

> **Why the DataChannel is created before createOffer()**
> The offer describes what will be opened. If no channel exists yet, the SDP has no data section and nothing useful is negotiated. So the "ready" branch calls `createDataChannel("file")` first.

6.7 createPeer() and forcing the relay

```
function createPeer() {
    if (pc) pc.close();
    const conn = new RTCPeerConnection({
        iceServers,
        iceTransportPolicy: new URLSearchParams(location.search).has("relay") ? "relay" : "all",
    });
    pc = conn;
    pc.onicecandidate = (e) => { if (e.candidate) send({ type: "candidate", candidate: e.candidate }); };
    pc.ondatachannel = (e) => setupChannel(e.channel);
    pc.onconnectionstatechange = () => {
        if (conn !== pc) return;
        if (conn.connectionState === "connected") findRoute(conn).catch(() => {});
        if (conn.connectionState === "failed") status("connection failed (different networks need a reachable TURN server)");
    };
}
```

- `onicecandidate` sends each address as soon as it is found (trickle ICE) - faster than waiting for all.
- `ondatachannel` fires on the side that did **not** create the channel (the receiver).
- `conn !== pc` ignores state changes from a connection that has already been replaced.
- Opening the page with `?relay=1` sets `iceTransportPolicy: "relay"`, so only TURN candidates are used. This is how you test that the relay really works.

6.8 findRoute(): direct or relay?

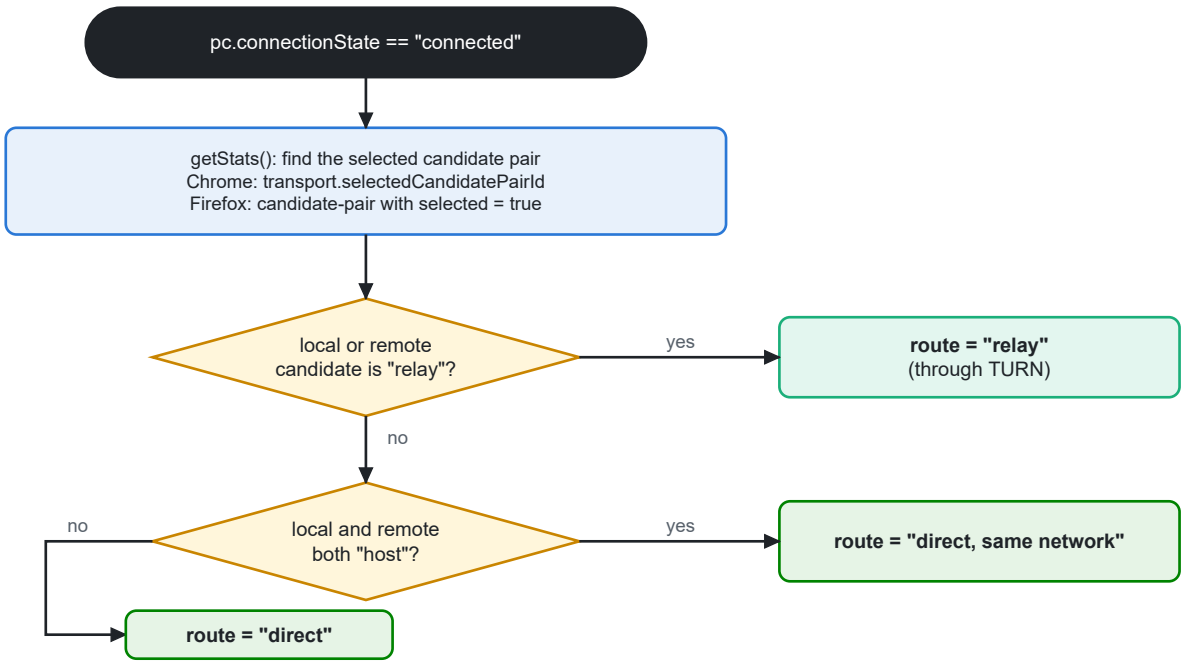


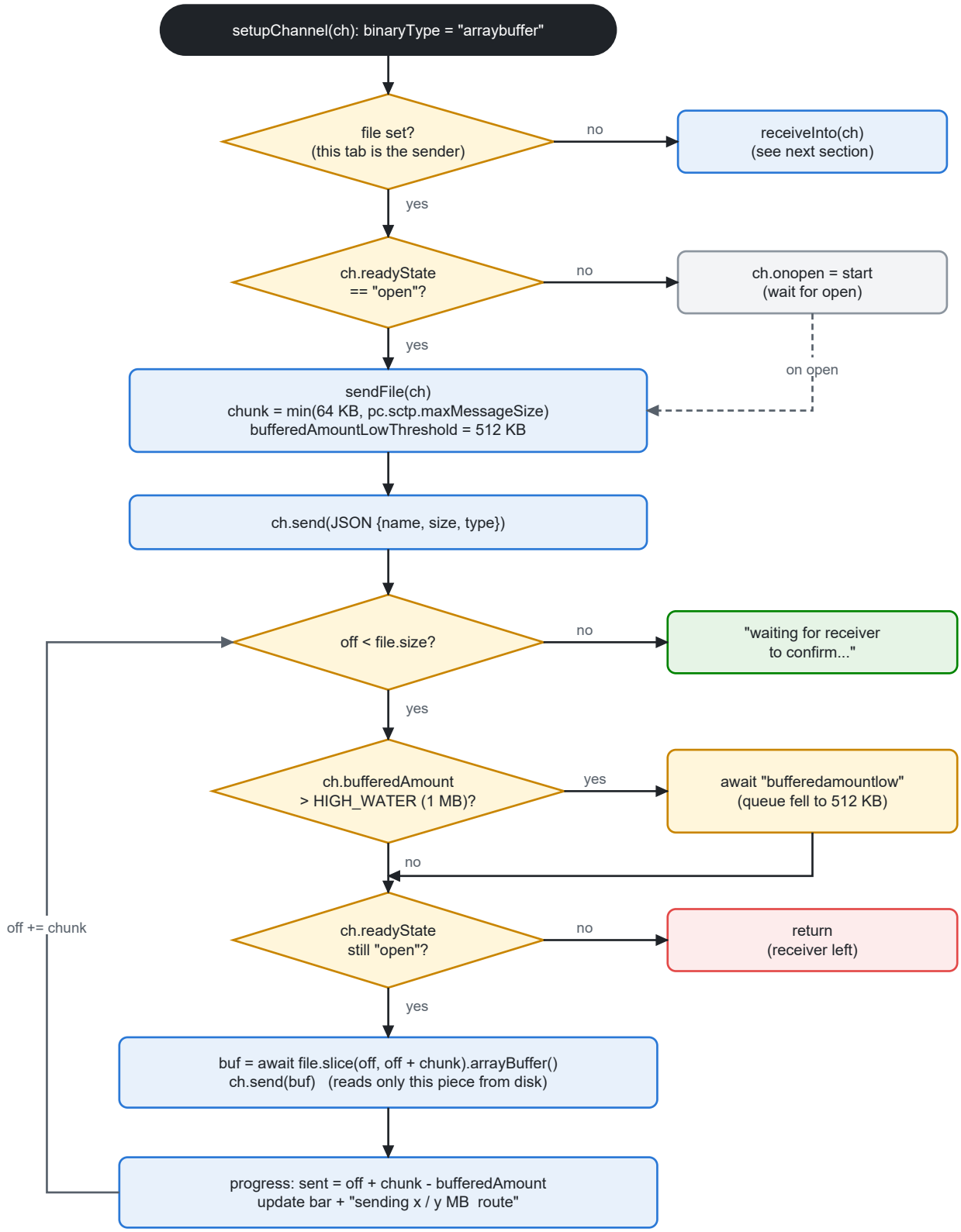
Figure 12. How the status line learns which path the data takes.

`getStats()` returns a report with the selected candidate pair. Chrome points to it from the `transport` entry; Firefox marks the pair itself with `selected`. The candidate types of both ends give the answer.

6.9 Sending: setupChannel() and sendFile()

```
function setupChannel(ch) {
    ch.binaryType = "arraybuffer";
    if (!file) return receiveInto(ch);
    const start = () => sendFile(ch).catch((err) => status("send failed: " + err.message));
    if (ch.readyState === "open") start(); else ch.onopen = start;
}
```

The role is decided by `file`, not by who created the channel - a sender sends whatever channel it gets. `binaryType = "arraybuffer"` makes binary messages arrive as ArrayBuffers instead of Blobs.



The sender never holds more than ~1 MB of the file in memory, whatever the file size.

Figure 13. sendFile() as a flowchart. The loop reads one chunk at a time and pauses whenever the send queue is above 1 MB.

Back-pressure, the most important idea in sendFile()

`ch.send()` does not wait for the network: it **queues** the data and returns immediately. Reading a 2 GB file in a tight loop would queue the whole file in memory and the browser would close the channel. So the loop checks `ch.bufferedAmount` (bytes queued but not yet sent). Above `HIGH_WATER` (1 MB) it waits for the `bufferedamountlow` event, which fires when the queue drops to `bufferedAmountLowThreshold` (512 KB).

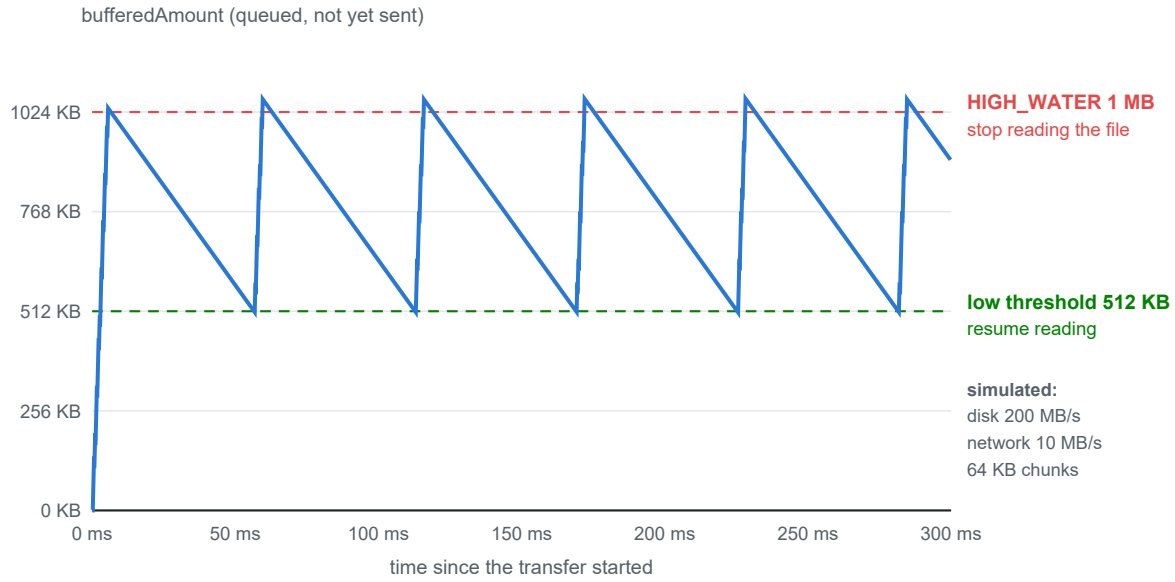


Figure 14. Simulated send queue during a transfer. Reading is fast, the network is slow, so the queue fills quickly, waits while it drains, then refills - a sawtooth between 512 KB and just over 1 MB.

- `file.slice(off, off + chunk).arrayBuffer()` reads only that piece from disk; the sender never loads the whole file.
- `Math.min(CHUNK, pc.sctp?.maxMessageSize || CHUNK)` respects a smaller message limit advertised by the other browser.
- Progress counts bytes **actually handed to the network**: `min(off + chunk, size) - bufferedAmount`.
- If the receiver disappears, `readyState` stops being `"open"` and the loop returns.
- When the receiver replies `"done"`, the sender calls `closePeer()` but keeps its WebSocket: the status becomes "sent ✓ code still active".

6.10 The DataChannel mini-protocol

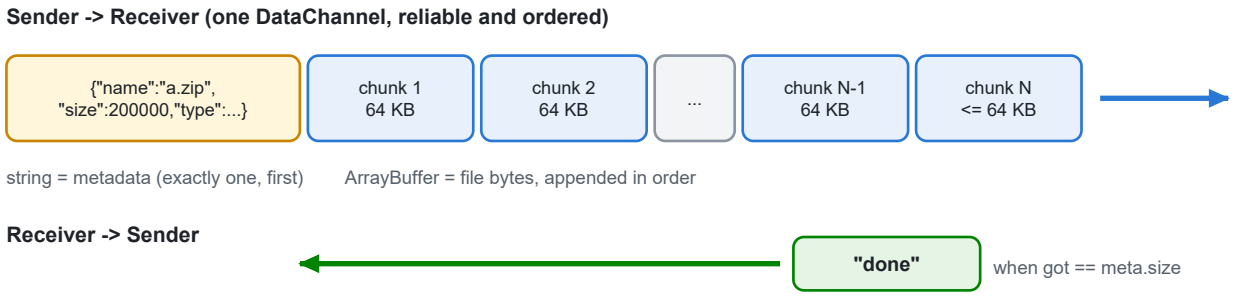


Figure 15. Everything that travels over the DataChannel. The message type (string vs binary) is the only framing needed.

6.11 Receiving: receiveInto() and save()

```
ch.onmessage = (e) => {
    if (typeof e.data === "string") { meta = JSON.parse(e.data); t0 = performance.now(); }
    else { parts.push(e.data); got += e.data.byteLength; }
    progress(got, meta.size);
    status(`receiving ${meta.name} ... (${speed}/s, ${route || "connecting"})`);
    if (got === meta.size) { save(meta, parts); parts = []; ch.send("done"); }
};
ch.onclose = () => {
    if (!meta || got < meta.size) status("transfer stopped: the sender picked another file, closed the tab or lost connection");
    leave();
};
```

- Because the channel is reliable and ordered, appending chunks in arrival order rebuilds the file exactly.
- An empty file works: `size` is 0, so the metadata message alone completes it.
- Speed is bytes received divided by the time since the metadata arrived.
- `onclose` fires either after "done" (the sender closed the connection - normal) or when the sender vanished mid-transfer. Either way the receiver leaves the room.

```
function save(meta, parts) {
    const a = document.createElement("a");
    a.href = URL.createObjectURL(new Blob(parts, { type: meta.type }));
    a.download = meta.name;
    a.textContent = "save " + meta.name;
    statusEl.replaceChildren("received ✓  ", a);
    a.click();
}
```

`save()` builds a Blob from the chunks and clicks a hidden download link. The link stays visible as "save <name>" because some phones block automatic downloads. File names are inserted with `textContent`, never `innerHTML`, so a malicious name cannot inject HTML.

> **Known limit: memory**
> All chunks stay in memory until the Blob is saved. That is fine up to about 1-2 GB on a laptop and less on a phone. The code marks this with a `ponytail:` comment; the fix (streaming to disk) is in chapter 14.

6.12 Cleanup and the "stale callback" pattern

```
function closePeer() { if (pc) pc.close(); pc = null; }
function leave() { joinId++; closePeer(); const socket = ws; ws = null; if (socket) socket.close(); }
```

`leave()` sets `ws = null` **before** closing the socket, so the socket's own `onclose` sees `socket !== ws` and stays quiet. The server then sends `bye` to the other side. This "is this callback still about the current thing?" check appears three times and is the backbone of the client's correctness:

Guard	Where	Prevents
`conn !== pc`	`onconnectionstatechange`	An old connection's "failed" overwriting the status of a new one
`socket !== ws`	`onmessage`, `onclose`	Messages or close events from a socket we already replaced
`id !== joinId`	`join()` after `await configReady`	A slow join reviving a code the user already abandoned

6.13 The page: index.html

Two `section` boxes and a progress bar. The Send box has `input type="file"`, the large `#code` display and a status line; the Receive box has the code input (`inputmode="numeric"`, `maxlength="6"`), the Receive button and its status line. `progress#bar` is hidden until a transfer starts. `link rel="icon" href="data:,"` is an empty icon that stops the browser requesting `/favicon.ico` and logging a 404. The script is loaded at the end of `body`, so every element exists when it runs.

Key idea
The client is a small state machine driven by three event sources - WebSocket messages, peer connection events, DataChannel events - and every handler first checks that the object it belongs to is still the current one.

7. Wire protocol reference

7.1 Signaling messages (WebSocket, JSON text)

Message	From -> to	Example / payload	Meaning
`ready`	server -> first peer	`{"type":"ready"}`	Second peer arrived; you make the offer
`offer`	sender -> receiver	`{"type":"offer","sdp":{type,sdp}}`	Session description of the offerer
`answer`	receiver -> sender	`{"type":"answer","sdp":{...}}`	Session description of the answerer
`candidate`	both ways	`{"type":"candidate","candidate":{candidate,sdpMid,...}}`	One ICE address
`bye`	server -> remaining peer	`{"type":"bye"}`	The other socket left the room
`full`	server -> 3rd socket	`{"type":"full"}`	Room already has two sockets (then close 1008)

7.2 DataChannel messages

Message	Direction	Type	Meaning
metadata	sender -> receiver	string (JSON)	`{"name":..., "size":..., "type":...}` - always first, exactly once
chunk	sender -> receiver	ArrayBuffer	Up to 64 KB of file bytes, in order
`"done"`	receiver -> sender	string	All bytes received and saved; sender may close

7.3 HTTP endpoints

Endpoint	Returns
`GET /`	`static/index.html`
`GET /static/*`	`app.js` and any other static file
`GET /config`	`{"iceServers": [...]}` for `new RTCPeerConnection()`
`WS /ws/{room_id}?role=send`	Signaling socket; `role=send` creates the room, no role joins it

8. Deployment: Docker, coturn and HTTPS

8.1 Two containers

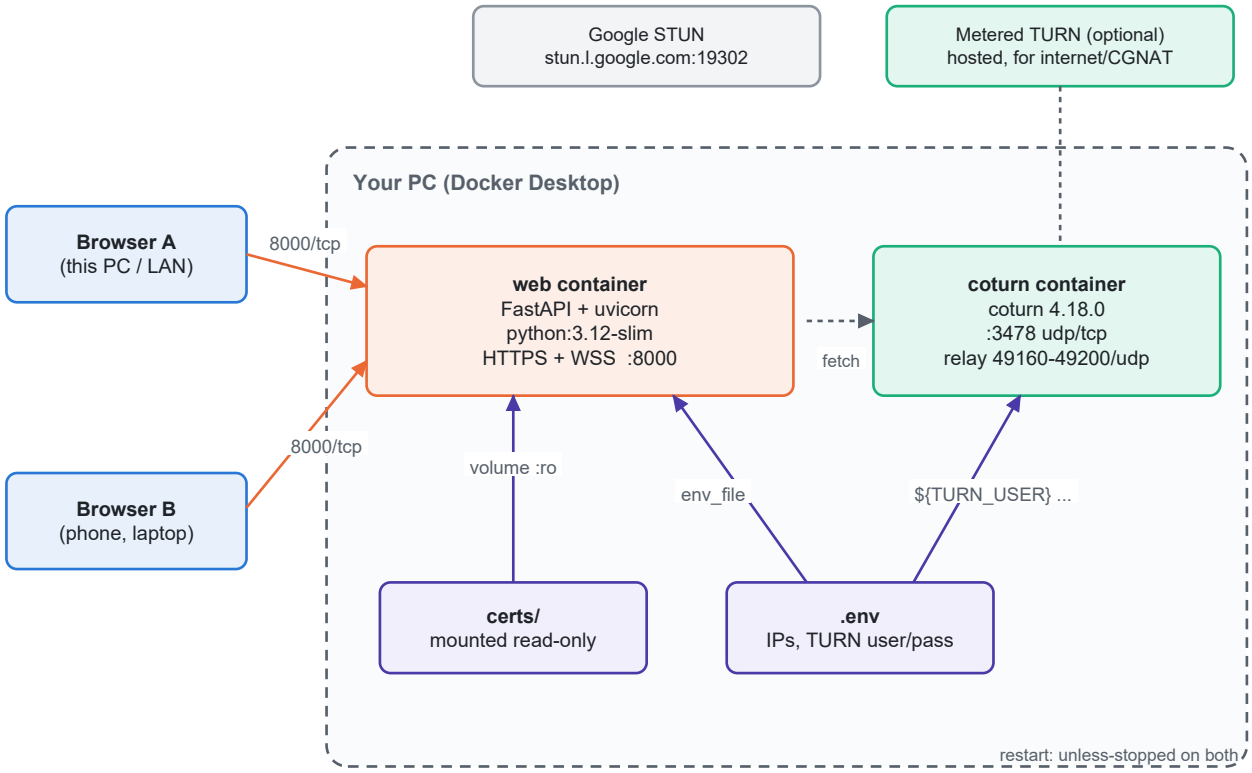


Figure 16. `docker compose up -d --build` starts two containers on your PC.

8.2 The web image (Dockerfile)

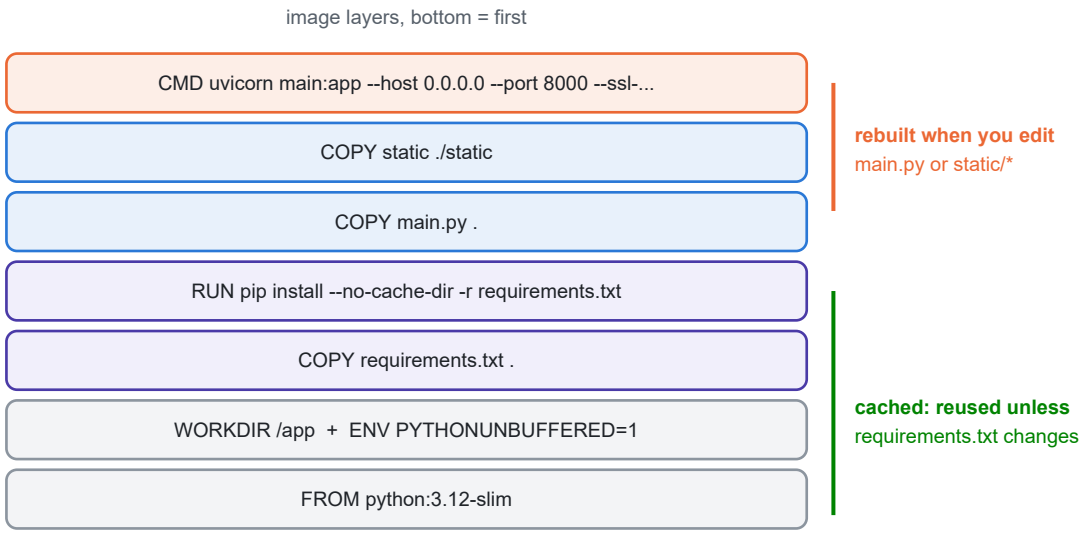


Figure 17. Layer order is chosen for caching: dependencies are installed before the code is copied.

- Copying `requirements.txt` and installing **before** copying the code means editing `app.js` does not reinstall FastAPI - Docker reuses the cached layers.
- `--host 0.0.0.0` listens on all interfaces; `127.0.0.1` would only accept connections from inside the container.

- `.dockerignore` keeps `certs/`, `.env`, `__pycache__` and Markdown files out of the image, so secrets are never baked in.
- Static files **are** baked in: after editing code, run `docker compose up -d --build` again and hard-refresh.

8.3 docker-compose.yml

Setting	Why
`web: build: .` + `ports: 8000:8000`	Build from the Dockerfile, publish HTTPS/WSS
`env_file: .env`	TURN_* and METERED_TURN_URL reach `main.py`
`dns: [1.1.1.1, 8.8.8.8]`	Hotspot DNS is flaky; `/config` must resolve the Metered API host
`volumes: ./certs:/app/certs:ro`	Certificate mounted read-only, never copied into the image
`restart: unless-stopped`	Both containers come back after a crash or reboot
`image: coturn/coturn:4.18.0`	**Pinned**: 4.18 removed `--no-dtls` and `--no-cli`, which made an unpinned setup crash on start
`3478/udp`, `3478/tcp`	STUN/TURN listening port on both protocols
`49160-49200/udp`	Relay ports; kept small because Docker Desktop publishes each port individually

8.4 coturn flags

Flag	Meaning
`-n`	No config file; everything is set by flags
`--listening-port=3478`	Standard STUN/TURN port
`--fingerprint`	Adds a checksum to STUN messages; browsers expect it
`--lt-cred-mech` + `--user=USER:PASS`	Require long-term username/password credentials
`--realm=webrtc.local`	Auth realm name; any string works
`--external-ip=${EXTERNAL_IP}`	**Important.** Inside Docker coturn only sees its 172.x container IP; this tells it which IP to advertise in relay candidates
`--min-port` / `--max-port`	Must match the published relay range 49160-49200
`--no-tls`	No TURN-over-TLS (would need a domain and cert); data is still DTLS-encrypted end to end
`--log-file=stdout`	Logs go to `docker compose logs coturn`

Harmless quirk: coturn 4.18 always logs `ERROR CONFIG: Unknown argument:` (with nothing after it) at start-up.

8.5 HTTPS and the certificate

Browsers only allow WebRTC and other powerful APIs on secure origins, and a LAN IP is not `localhost`, so the server runs HTTPS with a self-signed certificate whose `subjectAltName` lists the LAN IP, `127.0.0.1` and `localhost`. Each device shows a warning once; after "Advanced -> Proceed" the page and its `wss://` socket work. If the LAN IP changes (another Wi-Fi, a new hotspot address), regenerate the certificate and update `.env`.

```
MSYS_NO_PATHCONV=1 openssl req -x509 -newkey rsa:2048 -nodes \
  -keyout certs/key.pem -out certs/cert.pem -days 365 -subj "/CN=webrtc" \
  -addext "subjectAltName=IP:<YOUR_LAN_IP>,IP:127.0.0.1,DNS:localhost"
```

8.6 Running it

```
cp .env.example .env              # then set TURN_HOST, EXTERNAL_IP, TURN_PASS
docker compose up -d --build      # start web (:8000) and coturn (:3478)
docker compose logs -f web        # watch rooms being joined
docker compose down               # stop everything
```

Open `https://localhost:8000` on the PC and `https://<LAN_IP>:8000` on other devices. If another device cannot load the page, allow inbound TCP 8000 (and UDP 3478 and 49160-49200 for TURN) in Windows Firewall.

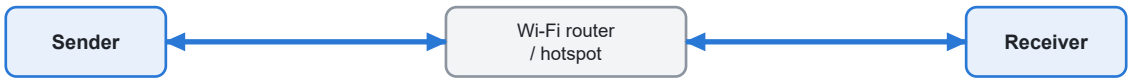
8.7 Over the internet

Option	Page + signaling	Relay	Works behind CGNAT?
Cloudflare quick tunnel + Metered	`cloudflared tunnel --url https://localhost:8000 --no-tls-verify` (new URL on every restart)	Metered hosted TURN (free 20 GB/month) via `METERED_TURN_URL`	Yes
Router port forwarding	Forward 8000/tcp; open `https://<PUBLIC_IP>:8000`	Forward 3478 tcp+udp and 49160-49200/udp; set TURN_HOST and EXTERNAL_IP to the public IP	No
Cloud VM	Run the same compose file on a VM with a public IP	coturn on the VM	Yes

Detecting CGNAT
If the WAN IP shown in your router differs from the IP "what is my ip" reports (common on mobile hotspots and Jio), your ISP uses carrier-grade NAT. Port forwarding cannot work; use a tunnel plus hosted TURN.

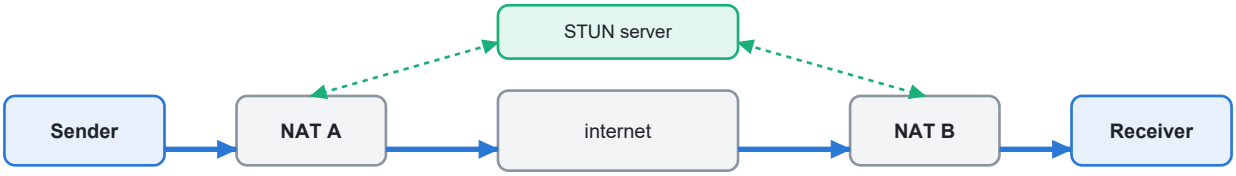
9. Networking in practice: LAN vs internet

A. Same Wi-Fi or hotspot -> host <-> host



LAN IPs (192.168.x.x) reach each other directly; the file never leaves the network.

B. Different home networks -> srflx <-> srflx (UDP hole punching)



STUN tells each side its public IP:port; both send at once so each NAT opens a pinhole.

C. CGNAT / strict NAT / mobile data -> relay (TURN)



Both sides connect OUT to the relay, which forwards the still-encrypted packets.
Uses relay bandwidth - that is why it is the last resort.

Figure 18. The three situations ICE ends up in, from best to last resort.

- **Same Wi-Fi or hotspot:** host candidates reach each other; the file never leaves the local network. A phone that *is* the hotspot counts as the same network as the laptop connected to it.
- **Different home networks:** host candidates fail; STUN-discovered `srflx` addresses plus simultaneous sending ("UDP hole punching") usually succeed. Still direct.
- **Strict NAT, mobile data, CGNAT:** hole punching fails; both sides connect out to TURN, which relays. The TURN server must be reachable from the internet - behind CGNAT your own coturn is not, so use Metered.
- Signaling (port 8000) must always be reachable by **both** browsers: automatic on a LAN, a tunnel or port forward over the internet.

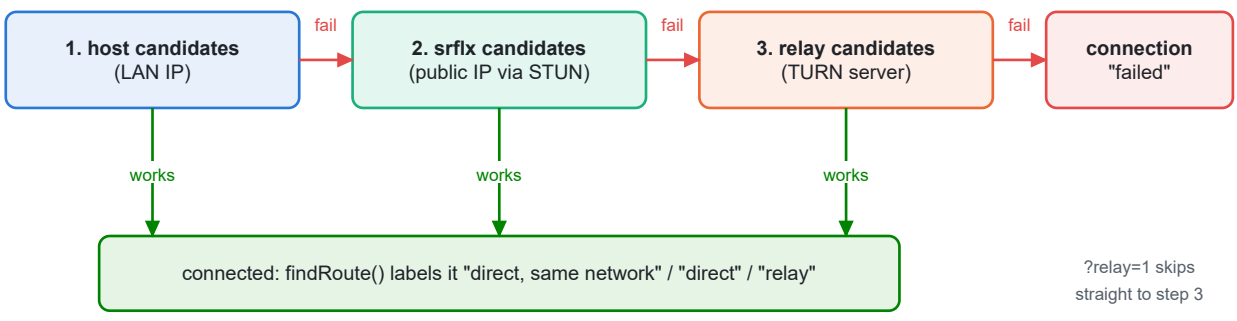


Figure 19. ICE tries every candidate pair, preferring direct paths; the relay is the fallback.

9.1 What speed to expect

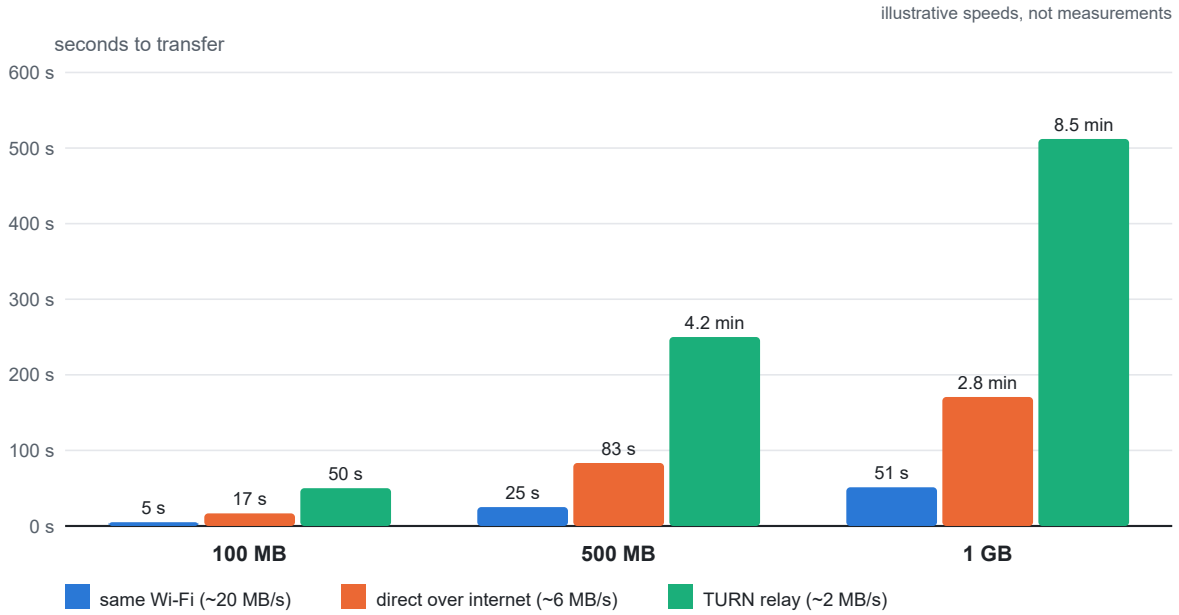


Figure 20. Rough transfer times by route. The speeds are typical assumptions for illustration, not measurements of this app; real numbers depend on Wi-Fi quality, upload bandwidth and the relay.

A relay path is slower because every byte makes two trips (sender -> TURN -> receiver) and is limited by the relay's bandwidth. Direct internet paths are limited by the slower side's **upload** speed. To see real numbers, watch the MB/s on the receiver's status line, or open `chrome://webrtc-internals`.

10. Security model

Party	Can see	Cannot see
FastAPI server	Room names (codes), SDP and ICE candidates (IP addresses), timing	Any file content, file name or size (those travel over the DataChannel)
TURN relay	Encrypted packets, sizes and timing, the two endpoints' addresses	File content - DTLS keys never leave the browsers
Network observer	That a TLS/DTLS session exists, traffic volume	Content of signaling or file
Anyone with the code	**The file** - while the sender's tab is open	Other codes

10.1 Threats and current mitigations

Threat	Today	Possible improvement
Guessing codes	1,000,000 codes, valid only while a sender tab is open; no rate limit	Rate-limit failed joins per IP; longer codes; sender approval prompt
Code busy / denial	Only one receiver at a time; a squatter could hold the room	Sender sees and can kick receivers; timeouts
TURN credential abuse	Static coturn user/pass is visible to anyone who loads `/config`	Time-limited TURN REST credentials (`--use-auth-secret`)
Malicious file name	Inserted via `textContent` and `a.download`; browsers sanitise download names	Show a confirm dialog with name and size before downloading
Man in the middle at signaling	TLS (self-signed: users must accept the warning once)	Real certificate (Let's Encrypt / named Cloudflare tunnel); compare SDP fingerprints
Secrets in the image	`.dockerignore` excludes `.env` and `certs/`	-

10.2 How likely are code collisions?

Each sender draws one of 1,000,000 codes. A collision with a code that is already active is the "birthday problem": with `n` active codes the chance that at least two senders drew the same one somewhere is about `1 - e^(-n(n-1)/2,000,000)`.

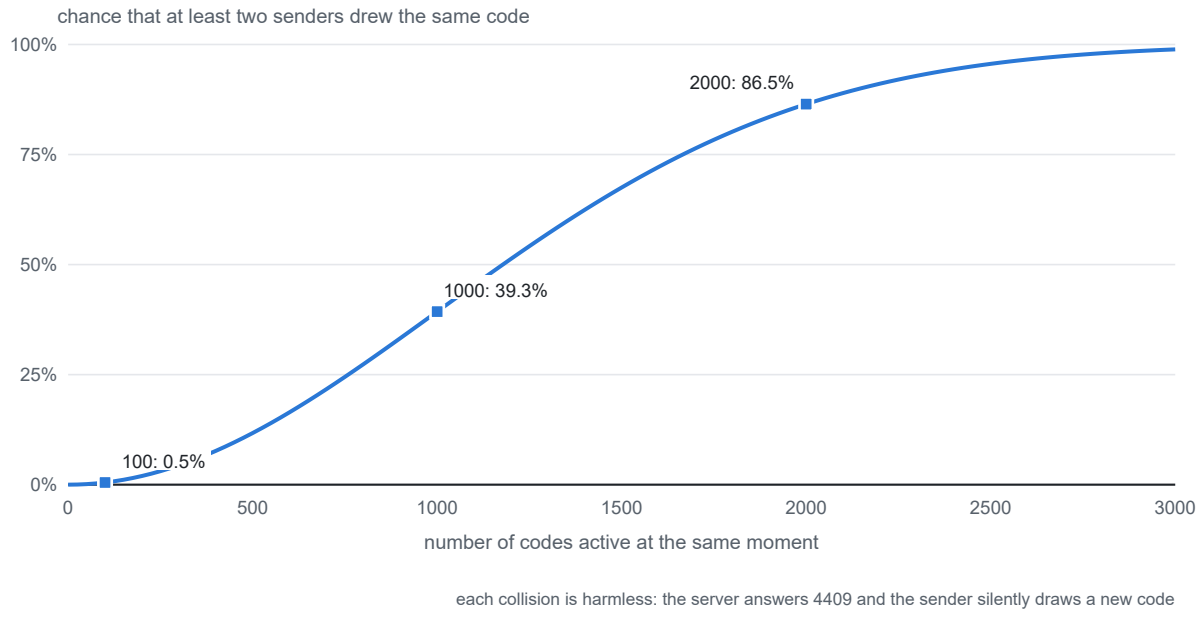


Figure 21. Probability that some pair of simultaneous senders drew the same code. For personal use (a handful of active codes) it is effectively zero, and the 4409 retry handles it anyway.

For guessing, the arithmetic is simple: with one active code, each random guess succeeds with probability one in a million. Without a rate limit a script can try many codes per second, so for public deployments the rate limit in chapter 14 matters more than code length.

> **Rule of thumb**
> Treat the code like a password: share it privately, and close the sender tab when you are done - that instantly invalidates the code.

11. Changes made: how the code evolved

There is no git history in this folder, so this chapter is **reconstructed** from three kinds of evidence: comments in the code that explain why something was added, the README, and the places where EXPLANATION.md (written earlier) no longer matches the code. Treat the "why" column as the stated or most likely reason.

11.1 Change log

#	Change	Why	Where / evidence
1	Senders own codes: `?role=send` creates the room; 4409 if taken (page draws a new code); 4404 for unknown codes; room deleted the moment the sender leaves	Old codes stayed usable and receivers could wait forever in rooms nobody would join	`signaling()`; `join().onclose`; comment "a code (room) exists only while its sender is connected"
2	Identity check `rooms.get(room_id) is peers` on cleanup	A late cleanup must not delete a newer room that reuses the code	`signaling()` finally block
3	Optional Metered hosted TURN, 1-hour cache, 60-second retry, fallback to coturn	Works across the internet and behind CGNAT; the Metered API takes seconds to answer	`metered_ice_servers()`, `.env.example`, README
4	`dns: [1.1.1.1, 8.8.8.8]` for the web container	Hotspot DNS was flaky; `/config` must reach Metered	compose comment
5	coturn pinned to 4.18.0; `command:` written as a YAML list	Unpinned image crashed (removed flags); list avoids stray empty arguments	compose; EXPLANATION.md
6	ICE config prefetched at page load (`configReady`) with retry on failure	Pressing Send/Receive no longer waits for `/config`	`app.js` comment
7	WebRTC warm-up: create and close an RTCPeerConnection after render	The first peer connection can take seconds to create	`app.js` comment
8	`joinId` generation counter	A join still waiting for config could revive an abandoned code	`app.js` comment
9	Route detection with `getStats()` shown in the status line	Lets users and testers see "direct, same network", "direct" or "relay"	`findRoute()`
10	`?relay=1` forces `iceTransportPolicy: "relay"`	Test the TURN path on a LAN	`createPeer()`
11	Back-pressure tightened from 8 MB / 4 MB to 1 MB / 512 KB	Less memory queued; progress reflects real network progress (likely reason)	EXPLANATION.md says 8 MB, code says 1 MB
12	Progress counts bytes actually sent (`... - bufferedAmount`)	Bar no longer runs ahead of the network	`sendFile()`
13	Sender keeps the code after "done"	Several receivers can fetch the same file one after another	`sendFile()` onmessage, README
14	Clearer status messages (expired / busy / transfer stopped)	Each failure tells the user what happened	`EXPIRED`, `join()`, `receiveInto()`

11.2 Where EXPLANATION.md is out of date

Topic	EXPLANATION.md says	Code actually does
Back-pressure	`HIGH_WATER` 8 MB, resume under 4 MB	1 MB, resume at 512 KB
Who goes first	"Either side can be first"; the receiver may make the offer	Receivers get 4404 if the room does not exist, so the sender is always first and always offers. Deciding by `file` in `setupChannel` remains as a safe generic rule
join()	Fetches `/config` inside `join()`	Uses the prefetched `configReady`, plus the `joinId` check
Not mentioned	-	Route detection, WebRTC warm-up, Metered cache, compose `dns`, `restart` policy
Messages	"code busy, try again in a moment"; "sender disconnected before the file finished"	"code busy, someone is receiving it, ..."; "transfer stopped: the sender picked another file, ..."
Troubleshooting	Wrong code leaves the receiver "waiting for sender..."	A wrong or expired code now fails immediately with the EXPIRED message (4404)

12. Testing

12.1 What has been tested

According to EXPLANATION.md, an automated test drove two headless Chrome tabs against the Docker containers:

Scenario	Verified
Send a random file	Receiver's file has the same SHA-256 hash as the original
Second receiver on the same code	Gets the file too, after the first one finished
Sender status	Shows "sent ✓" after the receiver confirms

Not covered yet: real phones, Firefox and Safari, transfers across the internet, and real-world speed. That test script is not in the repository.

12.2 Manual test checklist

Test	How	Expected
Same PC	Two tabs on `https://localhost:8000`	"direct, same network", hash matches
Phone on the same Wi-Fi	`https://<LAN_IP>:8000` on the phone	Direct, fast
Relay	Receiver opens the page with `?relay=1`	Status shows "relay"
Empty file	Send a 0-byte file	Saves immediately
Wrong code	Type 000000	EXPIRED message at once (4404)
Busy code	Third tab joins during a transfer	"code busy, someone is receiving it"
Sender leaves mid-transfer	Close the sender tab	Receiver: "transfer stopped..."
Receiver leaves mid-transfer	Close the receiver tab	Sender: "receiver left, waiting for receiver..."
Second receiver	After a success, join the same code again	Gets the file again
Large file	1 GB on a laptop	Completes; memory rises on the receiver only

12.3 A server test you could add

The signaling rules are pure logic and easy to test with FastAPI's `TestClient` (needs `httpx` and `pytest`). This sketch was run against the current main.py and both tests pass:

```
from fastapi.testclient import TestClient
from starlette.websockets import WebSocketDisconnect
import pytest
from main import app, rooms

client = TestClient(app)

def test_pair_relay_and_cleanup():
    with client.websocket_connect("/ws/file-111111?role=send") as s:
        with client.websocket_connect("/ws/file-111111") as r:
            assert s.receive_json() == {"type": "ready"}
            s.send_text('{"type":"offer"}')
            assert r.receive_text() == '{"type":"offer"}'
        assert s.receive_json() == {"type": "bye"}       # receiver left
    assert "file-111111" not in rooms                    # sender left -> room gone

def test_unknown_code_rejected():
    with pytest.raises(WebSocketDisconnect) as e:
        with client.websocket_connect("/ws/file-000000") as r:
            r.receive_text()
    assert e.value.code == 4404
```

13. Troubleshooting

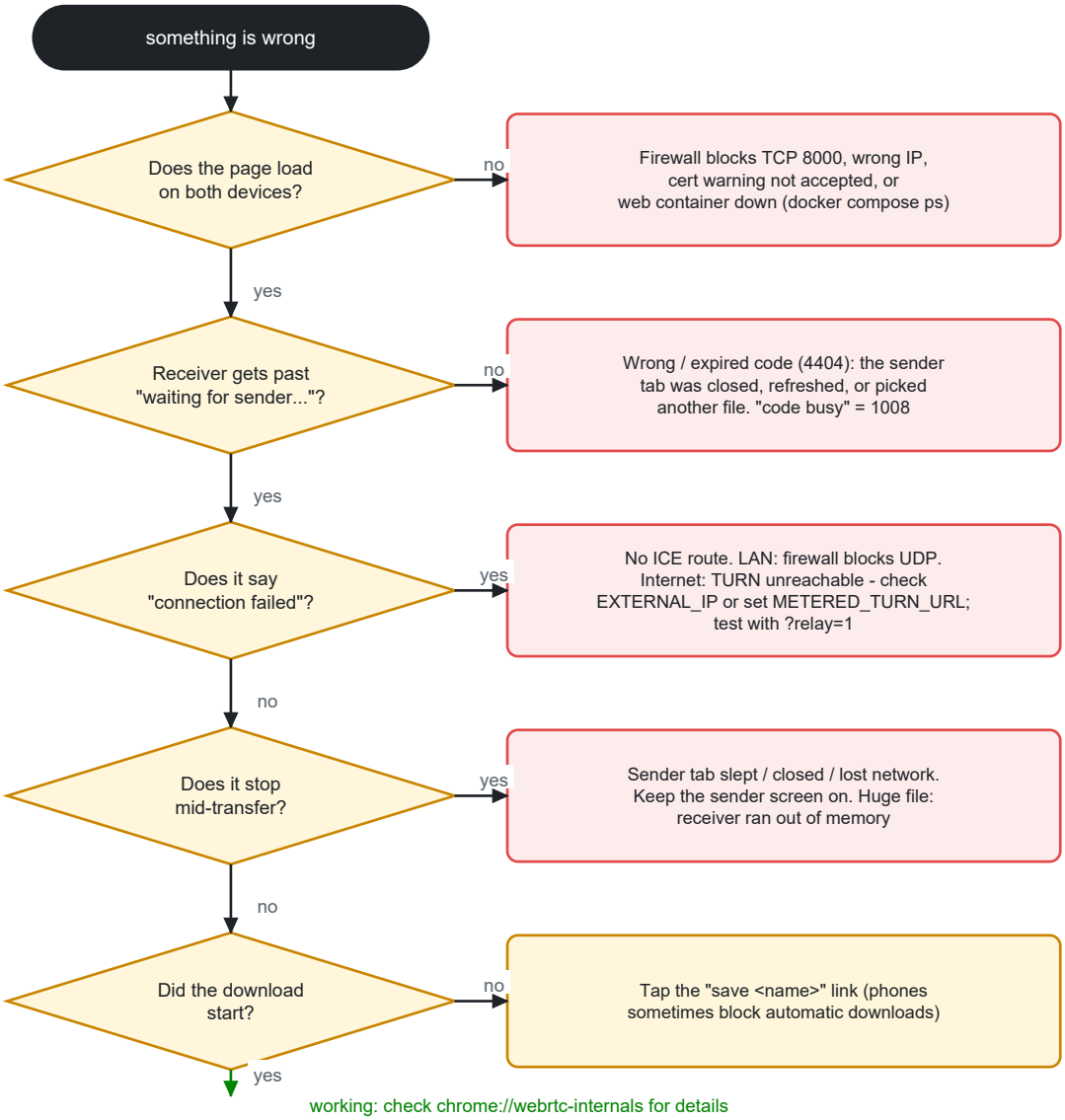


Figure 22. Work down the questions; the first "wrong" answer points at the cause.

Symptom	Likely cause / fix
Page does not load on another device	Windows Firewall blocks inbound TCP 8000, or the network profile is Public while Docker is only allowed on Private. Add an inbound rule
"code expired ... or wrong"	Typo, or the sender tab was closed, refreshed or picked another file
"code busy, someone is receiving it"	Another receiver is using the code; wait until they finish
"connection failed"	No ICE route. LAN: UDP blocked by firewall. Internet: TURN unreachable - check `EXTERNAL_IP` and port forwards, or set `METERED_TURN_URL`
"transfer stopped ..."	Sender tab slept, closed or lost network. Phones pause background tabs - keep the sender screen on
Download does not start on a phone	Tap the "save <name>" link
Receiving tab crashes on a huge file	Out of memory: the receiver holds the whole file (section 6.11)
"can't reach server" / "disconnected from server"	The web container is down: `docker compose ps`, `docker compose logs web`
Code changes do not show up	Files are baked into the image: `docker compose up -d --build`, then Ctrl+F5
coturn keeps restarting	An unsupported flag: `docker compose logs coturn`, look for "unrecognized option"

Useful browser pages: `chrome://webrtc-internals` (Chrome) and `about:webrtc` (Firefox) show every candidate, the selected pair and DataChannel byte counts.

14. Future aspects: where this can go

The current design is deliberately minimal. This chapter lists the natural next steps, roughly ordered by effort, with how each would fit into the existing code.

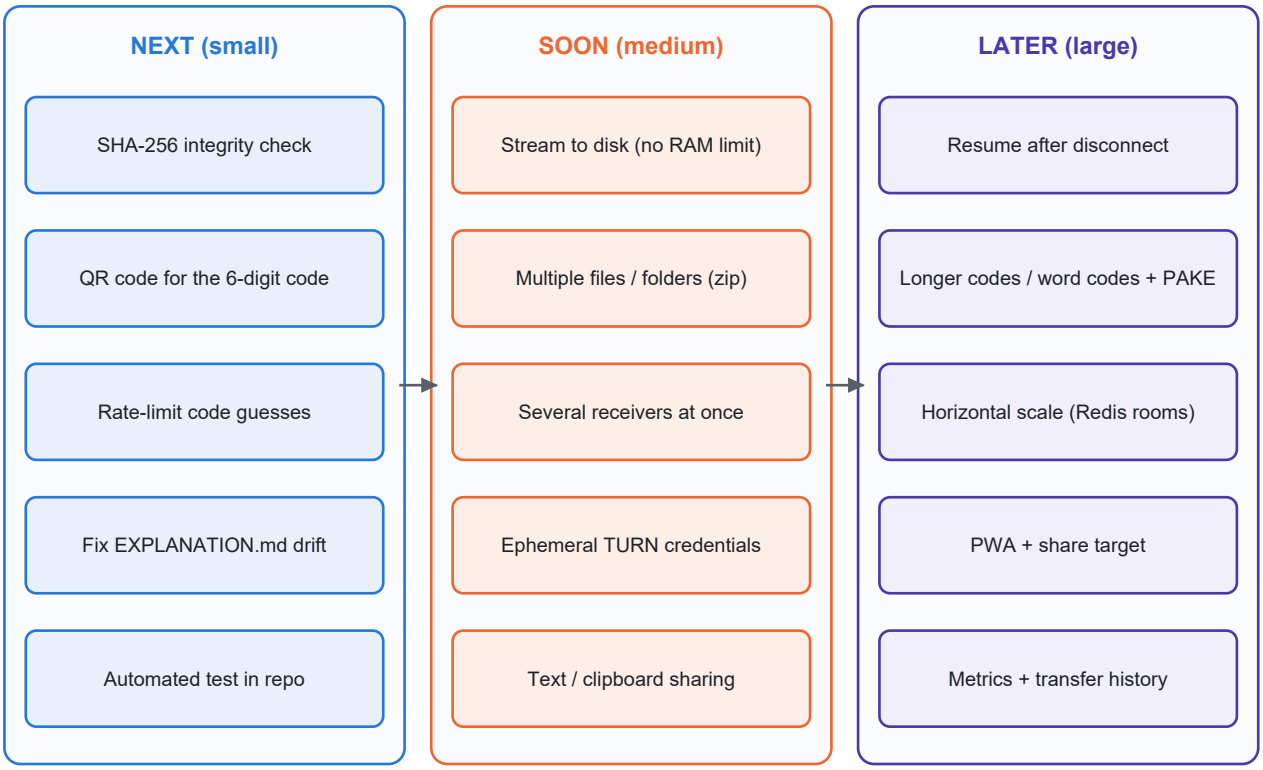


Figure 23. A possible roadmap.

14.1 Stream large files to disk

Problem: the receiver holds the whole file in memory. **Fix:** ask for a save location before the transfer and write each chunk as it arrives. Chrome and Edge support `showSaveFilePicker()`; other browsers need a service worker that turns chunks into a streamed download.

```
// in receiveInto(), after the metadata arrives (needs a user click to open the picker)
const handle = await showSaveFilePicker({ suggestedName: meta.name });
const out = await handle.createWritable();
// for each chunk:   await out.write(e.data);
// when got === size: await out.close(); ch.send("done");
```

14.2 Integrity check

The DataChannel is reliable and DTLS already protects packets in transit, but an end-to-end hash also catches bugs and storage errors. The sender includes a SHA-256 in the metadata (`crypto.subtle.digest`), the receiver compares before showing "received ✓". `crypto.subtle` has no streaming API, so very large files need per-chunk hashes or a small incremental hash library.

14.3 Multiple files and folders

Send a manifest first (`[{name, size, type}, ...]`), then the files back to back; or zip on the fly. `input type="file" multiple webkitdirectory` selects folders.

14.4 Several receivers at once

Today a room holds two sockets. For parallel receivers the sender becomes a hub with one RTCPeerConnection per receiver. The server would give each socket an id and route messages with `to`/`from` fields instead of forwarding to "the other one". Note that the sender's upload bandwidth is shared between receivers.

14.5 Resume after a disconnect

With streaming to disk in place, the receiver can rejoin and send `{"resumeFrom": got}`; the sender starts `file.slice()` at that offset. This turns a dropped phone connection from "start over" into a short pause.

14.6 Security hardening

- **Rate limiting:** count failed joins (4404) per client IP in `signaling()` and refuse after a threshold.
- **Longer or word-based codes** (e.g. "7-tiger-orbit"), optionally with a PAKE so the code also authenticates the peers, as Magic Wormhole does.
- **Sender approval:** show "someone wants this file - allow?" before sending.
- **Ephemeral TURN credentials:** run coturn with `--use-auth-secret --static-auth-secret=...`; `/config` then returns a username of the form `expiry:user` and an HMAC-SHA1 password that stop working after an hour.
- **Real certificates:** a named Cloudflare tunnel or Let's Encrypt removes the warning and the risk of users learning to click through it.

14.7 Scale and operations

- `rooms` lives in one process. To run several servers, move rooms to Redis (pub/sub for forwarding) or use sticky routing by code.
- Add a `/health` endpoint, structured logs and simple metrics: transfers started/finished, relay ratio, TURN bandwidth (relay traffic is what costs money).
- Add the automated two-browser test (Playwright) and the server test from section 12.3 to CI; extend to Firefox, Safari and real phones.

14.8 User experience

- A QR code containing `https://host/?code=123456` that pre-fills the Receive box.
- Drag and drop, an ETA next to the speed, dark mode.
- Installable PWA with the Web Share Target API, so "Share -> P2P File Share" works from a phone's gallery.
- Text and clipboard sharing over the same channel.

14.9 Housekeeping

Update EXPLANATION.md to match the code (chapter 11.2 lists the differences), and commit the project to git so future changes have a real history instead of a reconstructed one.

15. Learning aids

15.1 Hands-on exercises

1. Transfer a file between two tabs with `chrome://webrtc-internals` open. Find the selected candidate pair and its candidate types. Does it match the route in the status line?
2. Repeat with `?relay=1` on the receiver. Compare the MB/s and look at coturn's logs (`docker compose logs coturn`).
3. Change `CHUNK` to 16 KB and then 256 KB. What happens to speed, and why does the `maxMessageSize` check exist?
4. Set `HIGH_WATER` to 64 MB and send a large file. Watch the sender tab's memory in Task Manager.
5. Add `?code=123456` support: on load, if the URL has a code, fill the Receive box and click it.
6. Write the server tests from section 12.3 and run them with pytest.

15.2 Quiz

Q1. Why does the server send "ready" to only one socket?

Answer: So exactly one side creates the offer; two crossing offers ("glare") would break negotiation.

Q2. What would go wrong without the promise `chain`?

Answer: A candidate could be handled while the offer is still being applied; addIceCandidate would fail because there is no remote description yet.

Q3. Why is the DataChannel created before `createOffer()`?

Answer: The offer must describe the data channel; without it the SDP has no data section.

Q4. What does `bufferedAmount` measure, and why does the sender watch it?

Answer: Bytes queued in the browser but not yet sent. Without the limit, a big file would be queued entirely in memory and the channel would close.

Q5. A receiver types a code nobody is using. What happens?

Answer: The server closes the socket with 4404 and the page shows the "code expired ... or wrong" message.

Q6. Why does `leave()` set `ws = null` before calling `close()`?

Answer: So the socket's onclose sees socket !== ws and does not report an error for a close we caused.

Q7. Can the TURN server read the file?

Answer: No. It only relays DTLS-encrypted packets; keys are negotiated between the browsers and checked against the SDP fingerprint.

Q8. Why must coturn get `--external-ip`?

Answer: Inside Docker it only knows its 172.x container IP; relay candidates must carry an address browsers can reach.

Q9. Why is the server safe to run single-process without locks?

Answer: asyncio only switches tasks at await; the room check and creation have no await between them.

Q10. What keeps the code alive after a successful transfer?

Answer: The sender closes only the peer connection and keeps its WebSocket, so the room still exists.

15.3 Glossary

Term	Meaning
CGNAT	Carrier-grade NAT: the ISP shares one public IP among many customers; inbound connections are impossible
DataChannel	WebRTC peer-to-peer message pipe; reliable and ordered by default
DTLS	TLS over UDP; encrypts every DataChannel packet
Glare	Both peers sending an offer at the same time
ICE	Interactive Connectivity Establishment: gathers candidates and finds a working pair
ICE candidate	One possible address (IP, port, type: host, srflx, prflx, relay)
NAT	Network Address Translation: many private devices share one public address
Back-pressure	Slowing the producer when the consumer cannot keep up (here: bufferedAmount)
SCTP	Transport protocol under the DataChannel: message framing and flow control
SDP	Session Description Protocol: the offer/answer text blob
Signaling	Exchanging SDP and candidates through a side channel before connecting
srflx	Server-reflexive candidate: your public IP:port as seen by STUN
STUN	Server that tells a client its public address
TURN	Server that relays traffic when no direct path exists
Trickle ICE	Sending candidates one by one as they are discovered
WSS	WebSocket over TLS

Appendix A. Full source code

The complete source, with line numbers, as it is at the time of writing. `.env` and `certs/` are deliberately not included - they hold secrets.

A.1 main.py

```
 1  import json
 2  import os
 3  import time
 4  import urllib.request
 5  from pathlib import Path
 6
 7  from dotenv import load_dotenv
 8  from fastapi import FastAPI, WebSocket, WebSocketDisconnect
 9  from fastapi.responses import FileResponse
10  from fastapi.staticfiles import StaticFiles
11
12  BASE = Path(__file__).parent
13  load_dotenv(BASE / ".env")
14
15  TURN_HOST = os.getenv("TURN_HOST", "127.0.0.1")
16  TURN_USER = os.getenv("TURN_USER", "webrtc")
17  TURN_PASS = os.getenv("TURN_PASS", "webrtc")
18  # optional: Metered TURN credentials URL (with ?apiKey=...), for use over the internet
19  METERED_TURN_URL = os.getenv("METERED_TURN_URL")
20  metered_cache: tuple[float, list] = (0.0, [])  # (fetched at, iceServers)
21
22  app = FastAPI()
23  app.mount("/static", StaticFiles(directory=BASE / "static"), name="static")
24
25  rooms: dict[str, list[WebSocket]] = {}  # room id -> the (max 2) sockets in it
26
27
28  async def send_safe(ws: WebSocket, text: str):
29      # the other peer may be disconnecting right now; that must not kill this connection
30      try:
31          await ws.send_text(text)
32      except Exception:
33          pass
34
35
36  @app.get("/")
37  def index():
38      return FileResponse(BASE / "static" / "index.html")
39
40
41  def metered_ice_servers(url: str):
42      # the Metered API takes seconds to answer; its credentials don't change per request, so reuse them for an hour
43      global metered_cache
44      fetched_at, servers = metered_cache
45      if not servers or time.time() - fetched_at > 3600:
46          try:
47              with urllib.request.urlopen(url, timeout=5) as resp:
48                  servers = json.load(resp)
49              metered_cache = (time.time(), servers)
50          except Exception:
51              if not servers:
52                  raise
53              metered_cache = (time.time() - 3600 + 60, servers)  # keep the old credentials, retry in a minute
54      return servers
55
56
57  @app.get("/config")
58  def config():
59      # ICE servers handed to the browser, so the JS never hardcodes IPs or passwords
60      if METERED_TURN_URL:
```

```
        # Metered returns a ready-made iceServers list with TURN credentials
        try:
            return {"iceServers": metered_ice_servers(METERED_TURN_URL)}
        except Exception as err:
            print("Metered TURN failed, falling back to coturn:", err)
    return {
        "iceServers": [
            {"urls": "stun:stun.l.google.com:19302"},
            {"urls": f"stun:{TURN_HOST}:3478"},
            {
                "urls": [
                    f"turn:{TURN_HOST}:3478?transport=udp",
                    f"turn:{TURN_HOST}:3478?transport=tcp",
                ],
                "username": TURN_USER,
                "credential": TURN_PASS,
            },
        ]
    }


@app.websocket("/ws/{room_id}")
async def signaling(websocket: WebSocket, room_id: str, role: str = ""):
    await websocket.accept()

    # a code (room) exists only while its sender is connected: the sender creates it, receivers can only join it
    if role == "send":
        if room_id in rooms:
            await websocket.close(code=4409)  # code already taken: the sender picks another one
            return
        peers = rooms[room_id] = []
    elif room_id in rooms:
        peers = rooms[room_id]
    else:
        await websocket.close(code=4404)  # expired or wrong code
        return

    if len(peers) >= 2:
        await websocket.send_text(json.dumps({"type": "full"}))
        await websocket.close(code=1008)
        return

    peers.append(websocket)
    print(f"room {room_id}: {len(peers)} peer(s)")

    # second person arrived: tell the first one to start the call (only one side offers)
    if len(peers) == 2:
        await send_safe(peers[0], json.dumps({"type": "ready"}))

    try:
        while True:
            text = await websocket.receive_text()
            for peer in peers:
                if peer is not websocket:
                    await send_safe(peer, text)  # forward offer/answer/candidate as-is
    except WebSocketDisconnect:
        pass
    finally:
        peers.remove(websocket)
        # sender gone = code expires at once, even if a receiver is still connected
        if rooms.get(room_id) is peers and (role == "send" or not peers):
            del rooms[room_id]
        for peer in peers:
            await send_safe(peer, json.dumps({"type": "bye"}))
        print(f"room {room_id}: {len(peers)} peer(s)")
```

A.2 static/app.js

```
// P2P file transfer: the file goes browser -> browser over an RTCDataChannel.
// The server only relays the handshake (offer/answer/candidates); it never sees file bytes.

// ---------- state ----------
let ws;                         // WebSocket to the signaling server
let pc;                         // RTCPeerConnection to the other browser
let iceServers = [];            // STUN/TURN list, fetched from /config
let route = "";                 // how the browsers reached each other: "direct" or "relay"
let chain = Promise.resolve(); // runs signaling messages one at a time, in order
let file = null;                // sender: the file being shared; receiver: null
let statusEl;                   // status line of whichever side (send/receive) is active
let joinId = 0;                 // bumped by leave(), so a join still awaiting config can't revive an old code

const CHUNK = 64 * 1024;             // bytes per DataChannel message
const HIGH_WATER = 1024 * 1024;      // pause reading the file while this much is queued unsent

const $ = (id) => document.getElementById(id);
const status = (t) => (statusEl.textContent = t);
const mb = (n) => (n / 1048576).toFixed(1) + " MB";
const progress = (done, total) => {
    $("bar").hidden = false;
    $("bar").value = total ? done / total : 1;
};
// fetch ICE servers as soon as the page opens, so pressing Send/Receive doesn't wait for it
const loadConfig = () => fetch("/config").then((r) => r.json()).then((c) => c.iceServers);
let configReady = loadConfig();
// the first RTCPeerConnection in a browser can take seconds to create (WebRTC network setup);
// pay that right after the page renders, while the user is still picking a file or typing the code
setTimeout(() => new RTCPeerConnection().close(), 0);
const send = (obj) => {
    if (ws && ws.readyState === WebSocket.OPEN) ws.send(JSON.stringify(obj));
};

// ---------- peer connection ----------
function createPeer() {
    if (pc) pc.close();
    const conn = new RTCPeerConnection({
        iceServers,
        // open the page with ?relay=1 to force everything through TURN (for testing the relay)
        iceTransportPolicy: new URLSearchParams(location.search).has("relay") ? "relay" : "all",
    });
    pc = conn;
    pc.onicecandidate = (e) => {
        if (e.candidate) send({ type: "candidate", candidate: e.candidate });
    };
    // whoever joined the code second gets the channel from the other side
    pc.ondatachannel = (e) => setupChannel(e.channel);
    pc.onconnectionstatechange = () => {
        if (conn !== pc) return;
        if (conn.connectionState === "connected") findRoute(conn).catch(() => {});
        if (conn.connectionState === "failed") {
            status("connection failed (different networks need a reachable TURN server)");
        }
    };
}

// host = same network, srflx/prflx = direct through NAT, relay = through the TURN server
async function findRoute(conn) {
    const stats = await conn.getStats();
    let pair;
```

```
    stats.forEach((s) => {
        if (s.type === "transport" && s.selectedCandidatePairId) pair = stats.get(s.selectedCandidatePairId);
    });
    if (!pair) stats.forEach((s) => { if (s.type === "candidate-pair" && s.selected) pair = s; }); // Firefox
    const local = pair && stats.get(pair.localCandidateId);
    const remote = pair && stats.get(pair.remoteCandidateId);
    if (!local || !remote) return;
    route = local.candidateType === "relay" || remote.candidateType === "relay" ? "relay"
        : local.candidateType === "host" && remote.candidateType === "host" ? "direct, same network" : "direct";
}

// either side may have created the channel; what matters is who holds the file
function setupChannel(ch) {
    ch.binaryType = "arraybuffer";
    if (!file) return receiveInto(ch);
    const start = () => sendFile(ch).catch((err) => status("send failed: " + err.message));
    if (ch.readyState === "open") start();
    else ch.onopen = start;
}

// ---------- sending ----------
async function sendFile(ch) {
    const chunk = Math.min(CHUNK, pc.sctp?.maxMessageSize || CHUNK);
    ch.bufferedAmountLowThreshold = HIGH_WATER / 2;
    ch.onmessage = (e) => {
        if (e.data !== "done") return;
        closePeer(); // receiver has everything; stay on the code for the next receiver
        status(`sent ✓  code still active, keep this tab open to send again`);
    };

    ch.send(JSON.stringify({ name: file.name, size: file.size, type: file.type }));
    for (let off = 0; off < file.size; off += chunk) {
        if (ch.bufferedAmount > HIGH_WATER) {
            await new Promise((r) => ch.addEventListener("bufferedamountlow", r, { once: true }));
        }
        if (ch.readyState !== "open") return; // receiver left mid-transfer
        ch.send(await file.slice(off, off + chunk).arrayBuffer());
        const done = Math.max(0, Math.min(off + chunk, file.size) - ch.bufferedAmount); // actually sent
        progress(done, file.size);
        status(`sending ${mb(done)} / ${mb(file.size)}  ${route}`);
    }
    status("waiting for receiver to confirm...");
}

// ---------- receiving ----------
function receiveInto(ch) {
    let meta;
    let parts = [];
    let got = 0;
    let t0;

    ch.onmessage = (e) => {
        if (typeof e.data === "string") {
            meta = JSON.parse(e.data);
            t0 = performance.now();
        } else {
            parts.push(e.data);
            got += e.data.byteLength;
        }
        progress(got, meta.size);
```

```
        const speed = mb(got / ((performance.now() - t0) / 1000 || 1));
        status(`receiving ${meta.name}  ${mb(got)} / ${mb(meta.size)}  (${speed}/s, ${route || "connecting"})`);
        if (got === meta.size) {
            save(meta, parts);
            parts = [];
            ch.send("done"); // sender closes the connection when it gets this
        }
    };
    // fires after "done" (sender closed) or if the sender vanished mid-transfer
    ch.onclose = () => {
        if (!meta || got < meta.size) status("transfer stopped: the sender picked another file, closed the tab or lost
➜  connection");
        leave();
    };
}

// ponytail: whole file is held in memory until saved; fine up to ~1-2 GB on desktop, less on phones.
// For bigger files, stream to disk with showSaveFilePicker() (Chrome) or a service worker.
function save(meta, parts) {
    const a = document.createElement("a");
    a.href = URL.createObjectURL(new Blob(parts, { type: meta.type }));
    a.download = meta.name;
    a.textContent = "save " + meta.name;
    statusEl.replaceChildren("received ✓  ", a);
    a.click();
}

// ---------- signaling ----------
async function handle(msg) {
    if (msg.type === "ready") {
        // we were on the code first: we make the offer
        createPeer();
        setupChannel(pc.createDataChannel("file"));
        await pc.setLocalDescription(await pc.createOffer());
        send({ type: "offer", sdp: pc.localDescription });
        status("connecting...");
    } else if (msg.type === "offer") {
        createPeer();
        await pc.setRemoteDescription(msg.sdp);
        await pc.setLocalDescription(await pc.createAnswer());
        send({ type: "answer", sdp: pc.localDescription });
        status("connecting...");
    } else if (msg.type === "answer") {
        if (pc && pc.signalingState === "have-local-offer") await pc.setRemoteDescription(msg.sdp);
    } else if (msg.type === "candidate") {
        if (!pc) return;
        try {
            await pc.addIceCandidate(msg.candidate);
        } catch (err) {
            console.warn("bad candidate", err);
        }
    } else if (msg.type === "bye") {
        if (file) {
            // receiver left before finishing (after "done", pc is already null); keep waiting on the same code
            if (!pc) return;
            closePeer();
            status("receiver left, waiting for receiver...");
        } else {
            // the sender closed the tab or picked another file: the code is gone
            leave();
            status(EXPIRED);
```

```
        }
    }
}

const EXPIRED = "code expired (the sender picked another file or closed the tab) or wrong";

async function join(code) {
    const id = joinId;
    try {
        iceServers = await configReady;
    } catch (err) {
        configReady = loadConfig(); // retry on the next attempt
        status("can't reach server");
        return;
    }
    if (id !== joinId) return; // user picked another file / code meanwhile
    route = "";
    const proto = location.protocol === "https:" ? "wss" : "ws";
    const socket = new WebSocket(`${proto}://${location.host}/ws/file-${code}${file ? "?role=send" : ""}`);
    ws = socket;
    chain = Promise.resolve();
    socket.onmessage = (e) => {
        const msg = JSON.parse(e.data);
        chain = chain.then(() => socket === ws && handle(msg)).catch(console.error);
    };
    socket.onclose = (e) => {
        if (socket !== ws) return; // we closed it ourselves
        ws = null;
        closePeer();
        if (e.code === 4409) share(file); // sender's random code is taken: pick another
        else if (e.code === 4404) status(EXPIRED);
        else if (e.code === 1008) status("code busy, someone is receiving it, try again in a moment");
        else status("disconnected from server");
        if (file && e.code !== 4409) $("code").textContent = ""; // that code is dead now
    };
}

function closePeer() {
    if (pc) pc.close();
    pc = null;
}

function leave() {
    joinId++;
    closePeer();
    const socket = ws;
    ws = null;
    if (socket) socket.close();
}

// ---------- buttons ----------
$("file").onchange = () => {
    if ($("file").files[0]) share($("file").files[0]);
};

// new file = new code; leave() drops the old code on the server immediately
function share(f) {
    leave();
    file = f;
    statusEl = $("sendStatus");
```

```
    $("recvStatus").textContent = "";
    $("bar").hidden = true;
    // 6 random digits: easy to type on a phone
    const code = String(crypto.getRandomValues(new Uint32Array(1))[0] % 1e6).padStart(6, "0");
    $("code").textContent = code;
    status(`${f.name} (${mb(f.size)})  enter this code on the other device, keep this tab open`);
    join(code);
}

$("receive").onclick = () => {
    const code = $("codeIn").value.trim();
    if (!/^\d{6}$/.test(code)) {
        $("recvStatus").textContent = "enter the 6-digit code";
        return;
    }
    leave();
    file = null;
    statusEl = $("recvStatus");
    $("file").value = "";
    $("code").textContent = "";
    $("sendStatus").textContent = "";
    $("bar").hidden = true;
    status("waiting for sender...");
    join(code);
};
$("codeIn").onkeydown = (e) => {
    if (e.key === "Enter") $("receive").click();
};
```

A.3 static/index.html

```
<!doctype html>
<html>

<head>
    <meta charset="utf-8">
    <meta name="viewport" content="width=device-width, initial-scale=1">
    <title>P2P File Share</title>
    <link rel="icon" href="data:,">
    <style>
        body { font-family: sans-serif; margin: 16px; background: #f4f4f4; }
        section { background: #fff; border: 1px solid #ccc; border-radius: 6px; padding: 12px; margin-bottom: 12px; ma
→ x-width: 480px; }
        h4 { margin: 0 0 8px; }
        #code { font-size: 2em; font-weight: bold; letter-spacing: 6px; margin: 8px 0; }
        #codeIn { font-size: 1.2em; width: 8em; letter-spacing: 3px; }
        progress { width: 100%; max-width: 506px; }
        .status { color: #555; margin-top: 6px; word-break: break-all; }
    </style>
</head>

<body>
    <h3>P2P File Share</h3>

    <section>
        <h4>Send</h4>
        <input type="file" id="file">
        <div id="code"></div>
        <div class="status" id="sendStatus"></div>
    </section>

    <section>
        <h4>Receive</h4>
        <input id="codeIn" inputmode="numeric" maxlength="6" placeholder="code">
        <button id="receive">Receive</button>
        <div class="status" id="recvStatus"></div>
    </section>

    <progress id="bar" value="0" max="1" hidden></progress>

    <script src="/static/app.js"></script>
</body>

</html>
```

A.4 docker-compose.yml

```yaml
services:
  # FastAPI: serves the page + WebSocket signaling over HTTPS
  web:
    build: .
    restart: unless-stopped
    ports:
      - "8000:8000"
    env_file: .env
    dns: [1.1.1.1, 8.8.8.8]   # hotspot DNS is flaky; /config needs to reach the Metered API
    volumes:
      - ./certs:/app/certs:ro

  # coturn = our own STUN + TURN server
  coturn:
    image: coturn/coturn:4.18.0
    restart: unless-stopped
    ports:
      - "3478:3478/udp"
      - "3478:3478/tcp"
      - "49160-49200:49160-49200/udp"
    command:
      - -n
      - --listening-port=3478
      - --fingerprint
      - --lt-cred-mech
      - --user=${TURN_USER}:${TURN_PASS}
      - --realm=webrtc.local
      - --external-ip=${EXTERNAL_IP}
      - --min-port=49160
      - --max-port=49200
      - --no-tls
      - --log-file=stdout
```

A.5 Dockerfile

```dockerfile
FROM python:3.12-slim
WORKDIR /app
ENV PYTHONUNBUFFERED=1
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY main.py .
COPY static ./static
EXPOSE 8000
CMD ["uvicorn", "main:app", "--host", "0.0.0.0", "--port", "8000", "--ssl-keyfile", "certs/key.pem", "--ssl-certfile",
→  "certs/cert.pem"]
```

A.6 .env.example

```
# Address browsers use to reach coturn.
#   LAN only:  your PC's LAN IP (ipconfig)   e.g. 192.168.1.20
#   Internet:  your public IP (whatismyip)   + router port forwards
TURN_HOST=192.168.1.20
# Address coturn puts in its relay candidates (same value as TURN_HOST)
EXTERNAL_IP=192.168.1.20
TURN_USER=webrtc
TURN_PASS=change-me

# Optional: Metered TURN (needed over the internet / behind CGNAT). Free 20 GB/month at metered.ca.
# When set, /config uses Metered instead of coturn.
METERED_TURN_URL=https://<yourapp>.metered.live/api/v1/turn/credentials?apiKey=<key>
```

A.7 requirements.txt and .dockerignore

```
fastapi
uvicorn[standard]
python-dotenv
```

```
__pycache__/
certs/
.env
*.md
```